FoliaScan-Azure

Phase 1 — Build and validate the model locally

Goal

The goal of Phase 1 is to build and verify a complete local machine-learning workflow before introducing Azure infrastructure.

Working locally first reduces cost and complexity and makes problems easier to debug. By the end of this phase, the data preparation, model pipeline, training process, checkpoint selection, and final evaluation should work reliably and reproducibly.

Complete workflow

```
Set up a reproducible Python project
        ↓
Prepare and validate the dataset
        ↓
Create leakage-safe data splits
        ↓
Load data through a manifest
        ↓
Build the PyTorch data and model pipeline
        ↓
Train and validate a baseline model
        ↓
Select the best checkpoint
        ↓
Evaluate it on the untouched test set
        ↓
Analyse model errors
```

Main concepts

1. Data leakage

Data leakage occurs when information from validation or test data indirectly influences model training, making evaluation results unrealistically good.

In FoliaScan, related images from the same physical leaf must remain in the same split:

```
One leaf_id → one final split
```

The general lesson is that data should be split using the true independent unit, such as a patient, customer, device, video, or physical object.

2. Train, validation, and test splits

Each split has a different role:

- **Train:** updates model parameters.
- **Validation:** compares epochs and selects the best model.
- **Test:** evaluates the final selected model once.

The test set must not influence hyperparameters, early stopping, checkpoint selection, or model architecture.

3. Manifest-driven datasets

A manifest is a structured file that describes the dataset, including information such as:

```
relative_path
class_name
split
leaf_id
source_split
```

The images remain unchanged, while the manifest determines how they are used.

This makes the workflow easier to audit, reproduce, version, and move between local and cloud environments.

4. Reproducibility

A reproducible workflow records enough information to recreate and compare experiments.

Important elements include:

- Version-controlled code
- Locked dependency versions
- Fixed random seeds
- Deterministic class mappings and splits
- Saved configurations
- Recorded metrics
- Saved checkpoints

Reproducibility does not always mean completely identical GPU results, but uncontrolled differences should be minimised and important settings should be recorded.

5. End-to-end validation

Individual functions may work correctly while the complete system still fails.

Phase 1 therefore validates the full connection:

```
data → transforms → batches → model → training → checkpoint → evaluation
```

Key takeaway

The purpose of Phase 1 is not only to train a classifier. It is to create a trustworthy local ML workflow that can later be moved to Azure without changing its fundamental data, training, and evaluation logic.

- 1.1 Project foundation

Objective

Create a structured and reproducible Python project in which dependencies, source code, tests, configurations, and development tools are organised before implementing the ML pipeline.

What was implemented

FoliaScan was created as a Python 3.11 project using:

- Poetry for project and dependency management
- A src-based package layout
- pytest for testing
- Ruff for linting
- mypy for static type checking
- Feature branches and pull requests for controlled development

These are the actual foundations described for subphase 1.1 in the original notes.

Concepts I should understand

1. Dependency management

A project depends on external packages such as PyTorch, Pillow, pandas, and Azure libraries.

Dependency management records:

- Which packages the project needs
- Which versions are compatible
- Which packages are required only for development

Dependencies are commonly divided into:

- **Runtime dependencies:** required to run the application
- **Development dependencies:** required for testing, linting, typing, or development

2. Poetry and virtual environments

Poetry manages the Python project and creates an isolated virtual environment.

The virtual environment prevents FoliaScan's dependencies from conflicting with packages used by other projects.

```
poetry install
poetry run <command>
```

- poetry install installs the project dependencies.
- poetry run executes a command inside FoliaScan's environment.

3. pyproject.toml and poetry.lock

pyproject.toml describes the project and its dependency requirements.

It may include:

- Project metadata
- Python version
- Runtime dependencies
- Development dependencies
- Tool configurations

poetry.lock records the exact package versions selected by Poetry.

```
pyproject.toml → allowed dependency versions
poetry.lock    → exact resolved versions
```

The lock file helps local machines, CI/CD systems, and deployment environments install consistent dependencies.

4. The src layout

The project package is stored under:

```
src/
└── foliascan/
```

A simplified structure is:

```
foliascan-azure/
├── src/foliascan/
├── tests/
├── configs/
├── docs/
├── data/
├── infra/
├── pyproject.toml
└── poetry.lock
```

The src layout separates application code from project configuration and reduces the risk of importing code accidentally from the repository root instead of from the installed package.

5. Testing, linting, and type checking

These tools detect different categories of problems:

Tool	Main responsibility
pytest	Checks program behaviour
Ruff	Detects code-quality and style problems
mypy	Checks type consistency

Examples:

- pytest can verify that a splitting function keeps one leaf in one split.
- Ruff can find unused imports or common coding problems.
- mypy can detect incorrect value types before runtime.

Passing one tool does not replace the others.

6. Feature branches and pull requests

A feature branch isolates one logical change from the stable main branch.

```
main
  └── feat/new-feature
```

This makes changes easier to:

- Review
- Test
- Reverse
- Merge safely
- Validate later with CI/CD

Development should normally happen on a feature branch rather than directly on main.

Important note

Never commit:

- Passwords or API keys
- Azure credentials
- Real .env files
- Raw datasets
- Large generated artifacts
- Temporary training outputs

These files may contain sensitive information, consume unnecessary repository space, or be generated again from code.

Essential commands

Install the environment

```
poetry install
```

Run quality checks

```
poetry run pytest
poetry run ruff check .
poetry run mypy src
```

Start a feature branch

```
git switch main
git pull --ff-only origin main
git switch -c feat/descriptive-name
```

Review and commit changes

```
git status
git diff
git add <files>
git diff --staged
git commit -m "feat: describe the change"
```

Review questions

1. What is the difference between runtime and development dependencies?
2. What different information is stored in pyproject.toml and poetry.lock?
3. Why is an isolated virtual environment useful?
4. What different problems do pytest, Ruff, and mypy detect?
5. Why is a src-based layout useful?
6. Why should development happen on feature branches?
7. Why must credentials and generated artifacts be excluded from Git?

Key takeaway

A professional ML project needs more than model code. Dependency management, isolated environments, clear source organisation, automated checks, and version-control practices make the project reproducible and maintainable.

- 1.2 Generic dataset preparation

Objective

Create a reusable pipeline for discovering, validating, describing, and splitting folder-based image datasets before model training.

What was implemented

The generic dataset tools can:

- Discover class folders and image files.
- Validate that images can be opened.
- Report corrupted or unsupported files.
- Summarise image formats, dimensions, and colour modes.
- Count the number of images in each class.
- Create deterministic train, validation, and test assignments.
- Save the results as structured metadata.

Concepts I should understand

1. Raw data and processed metadata

Raw data contains the original image files:

```
data/raw/
```

Processed metadata describes those files:

```
data/processed/dataset_manifest.csv
data/processed/dataset_report.json
```

The raw images remain unchanged. Reports and manifests record how the dataset was validated and organised.

2. Image validation

An image file may exist but still be unusable because it is:

- Corrupted
- Empty
- Unsupported
- Incorrectly encoded
- Impossible to decode

Dataset validation detects these problems before training starts.

A long training job should not fail because one invalid image was discovered too late.

3. Dataset inspection

Inspection helps answer questions such as:

- How many classes exist?
- How many images belong to each class?
- Are some classes much larger than others?
- Are image formats and dimensions consistent?
- Are any files corrupted?

Inspection does not modify the dataset. It creates information needed to make better training decisions.

4. Class imbalance

Class imbalance occurs when some classes contain many more examples than others.

This matters because a model may achieve good overall accuracy by performing well on large classes while ignoring smaller classes.

At this stage, the important task is to detect and report imbalance. Possible solutions can be considered later during training and evaluation.

5. Deterministic dataset preparation

A fixed random seed makes the same split reproducible:

```
random_seed = 42
```

The same data, code, and seed should generate the same assignments. This makes model comparisons more reliable.

Important note

The generic folder-based splitter works at the individual-image level. It is suitable only when images can be treated as independent samples.

It is not used for PlantVillage because PlantVillage contains related images identified by leaf_id.

Essential commands

```
# Inspect a folder-based dataset
poetry run python -m foliascan.data.cli inspect `
  --data-dir data/raw/plantvillage_tomato_color# Create a generic deterministic mani
poetry run python -m foliascan.data.cli split `
  --data-dir data/raw/plantvillage_tomato_color `
  --output data/processed/dataset_manifest.csv
```

Review questions

1. Why should images be validated before training?
2. What is the difference between raw data and processed metadata?
3. How can class imbalance make accuracy misleading?
4. Why should dataset preparation be reproducible?
5. When is an individual-image split unsafe?

Key takeaway

Dataset preparation should be an automated and repeatable pipeline, not a manual one-time task. Validating the data early prevents silent errors from affecting training and evaluation.

- 1.3 PlantVillage ingestion & safe splitting

Objective
Import the official PlantVillage Tomato dataset and create leakage-safe train, validation, and test assignments based on physical leaf groups.

What was implemented
The PlantVillage workflow:

- Loads mohanty/PlantVillage from Hugging Face.
- Uses the colour-image configuration.
- Filters the dataset to Tomato images.
- Exports ten tomato-condition classes.
- Creates a source manifest.
- Preserves the official test records.
- Groups related images using leaf_id.
- Creates the final FoliaScan manifest.
- Resolves leaf groups appearing across source splits.

Concepts I should understand

1. Dataset-specific ingestion

Generic tools cannot always handle the structure of an external dataset correctly.

PlantVillage contains useful metadata such as:

```
crop
disease
leaf_id
source split
```

The ingestion pipeline uses this metadata instead of treating the dataset as a simple collection of unrelated images.

2. Source manifest

The source manifest records the original information obtained from PlantVillage:

```
relative_path
class_name
source_split
leaf_id
```

It preserves where each image came from before FoliaScan applies its own final split rules.

3. Final FoliaScan manifest

The final manifest adds:

```
split
```

The difference is:

- source_split: the original PlantVillage assignment
- split: the final leakage-safe FoliaScan assignment

Keeping both columns makes the transformation auditable.

4. Group-aware splitting

The splitting unit is the physical leaf, not the individual image.

```
leaf_id = 500
├── image_1.jpg
├── image_2.jpg
└── image_3.jpg
```

All images belonging to this leaf_id must receive the same final split.

This prevents the model from seeing one view of a leaf during training and another view of the same leaf during evaluation.

5. Test precedence

Some related leaf records appeared across the official training and test splits.

FoliaScan applies this rule:

When a leaf_id appears in the official test set, every record with that leaf_id is assigned to the final test set.

The original source_split remains stored, so the change can still be inspected.

6. Integration testing of external datasets

Documentation describes how a dataset is expected to behave. The real records may contain unexpected cases.

External datasets should therefore be checked for:

- Missing metadata
- Duplicate identifiers
- Unexpected classes
- Invalid files
- Groups appearing across multiple splits

Important note

The generic FoliaScan split command must not be used for PlantVillage because it does not understand leaf_id relationships.

Essential commands

```
# Export the official Tomato subset
poetry run python -m foliascan.data.cli plantvillage-export `
  --output-dir data/raw/plantvillage_tomato_color `
  --source-manifest data/processed/plantvillage_source_manifest.csv# Create the leak
poetry run python -m foliascan.data.cli plantvillage-split `
  --source-manifest data/processed/plantvillage_source_manifest.csv `
  --output data/processed/dataset_manifest.csv `
  --validation-ratio 0.15 `
  --random-seed 42
```

Review questions

1. What does leaf_id represent?
2. Why is the generic image-level split unsafe for PlantVillage?
3. What is the difference between source_split and split?
4. Why is the official test split preserved?
5. Why does FoliaScan use test precedence?

Key takeaway

Splitting logic must reflect how the data was generated. For PlantVillage, the physical leaf is the independent unit, so all related images must remain together.

- 1.4 PyTorch data pipeline & smoke test

Objective

Connect the manifest, image loading, transformations, batching, and ResNet18 model into one working pipeline before full training.

What was implemented

The pipeline includes:

```
Manifest
   ↓
PyTorch Dataset
   ↓
Image transforms
   ↓
DataLoader
   ↓
ResNet18
   ↓
Class scores
```

It supports:

- Image loading and RGB conversion
- Deterministic class mapping
- Training augmentation
- Validation and test transforms
- ImageNet normalisation
- Train, validation, and test DataLoaders
- A ResNet18 classifier with ten outputs
- A forward-pass smoke test

Concepts I should understand

1. PyTorch Dataset

A Dataset defines how one sample is loaded.

```
Read one manifest row
        ↓
Locate the image
        ↓
Open and convert to RGB
        ↓
Apply transforms
        ↓
Return image tensor and class index
```

The Dataset answers:

How do I load one example?

2. PyTorch DataLoader

A DataLoader retrieves Dataset samples and combines them into batches.

```
32 individual samples → one batch
```

It also manages:

- Batch size
- Shuffling
- Batch order
- Parallel loading

The DataLoader answers:

How do I efficiently provide samples to the model?

3. Image transforms

Transforms prepare images for the model.

Training transforms include small random changes such as:

- Horizontal flipping
- Small rotations
- Conservative crop variation

Validation and test transforms are deterministic:

- Resize
- Centre crop
- Convert to tensor
- Normalise

Random augmentation is used only during training. Evaluation should not change randomly between runs.

4. ImageNet normalisation

ResNet18 was pretrained on ImageNet using a specific input normalisation.

Using the corresponding normalisation makes FoliaScan images more compatible with the pretrained model.

5. Class mapping

The model uses integer labels rather than class names:

```
Tomato___Bacterial_spot → 0
Tomato___Early_blight   → 1
...
Tomato___healthy        → 9
```

The same mapping must be used during training, validation, testing, and inference.

It is saved in the checkpoint so predictions can later be converted back to class names.

6. Tensor shapes

A typical image batch has the shape:

```
[32, 3, 224, 224]
```

This means:

```
32  images
3   RGB channels
224 image height
224 image width
```

For ten classes, the model output is:

```
[32, 10]
```

Each image receives ten class scores.

7. Transfer learning

ResNet18 has already learned useful general visual features from ImageNet.

FoliaScan replaces the original 1,000-class output layer with a ten-class layer:

```
Pretrained feature extractor
            ↓
New FoliaScan classification layer
```

This provides a practical baseline without training an entire neural network from random initialisation.

8. Smoke test

A smoke test is a small end-to-end integration check.

It verifies that:

- Manifest paths are correct
- Images can be loaded
- Transforms work
- Labels are valid
- A batch can be created
- The model accepts the batch
- Output dimensions are correct

It does not update model parameters.

Important note

A unit test checks a small component in isolation. A smoke test checks whether several real components work correctly together.

Essential command

```
poetry run python -m foliascan.training.smoke_test `
  --manifest data/processed/dataset_manifest.csv `
  --data-dir data/raw/plantvillage_tomato_color `
  --config configs/training.example.yaml `
  --split train `
  --device cuda
```

Review questions

1. What is the difference between a Dataset and a DataLoader?
2. What does [32, 3, 224, 224] represent?
3. Why must all splits share the same class mapping?
4. Why is random augmentation used only for training?
5. What does a smoke test verify?
6. Why was the final ResNet18 layer replaced?

Key takeaway

Before spending time on training, verify that the complete path from manifest to model

output works correctly.

- 1.5 Local baseline training

Objective

Turn the working data and model pipeline into a reproducible training workflow that produces checkpoints and training history.

What was implemented

The training workflow includes:

- CPU or CUDA device selection
- Fixed random seeds
- Training and validation loops
- Cross-entropy loss
- AdamW optimisation
- Best and last checkpoints
- Early stopping
- CSV and JSON training history
- Output-directory protection
- A command-line training interface

The test split is not used during training or checkpoint selection.

Concepts I should understand

1. Forward pass

The image batch passes through the model:

```
image batch → ResNet18 → class scores
```

The raw output scores are called logits.

2. Cross-entropy loss

CrossEntropyLoss compares the model's class scores with the correct class labels.

It is suitable because each image belongs to one of ten mutually exclusive classes.

The loss gives the training process a numerical measure of prediction error.

3. Backpropagation

Backpropagation calculates gradients for trainable model parameters.

Gradients indicate how the parameters should change to reduce the loss.

4. AdamW optimiser

AdamW uses the gradients to update model parameters.

It also applies weight decay, which can help limit overfitting by discouraging unnecessarily large parameter values.

5. Training and validation loops

During training:

- Gradients are calculated.
- Parameters are updated.
- Random augmentation may be used.

During validation:

- Gradients are disabled.
- Parameters remain unchanged.
- Deterministic transforms are used.
- Generalisation is measured.

6. Epoch

One epoch means the model has processed the complete training dataset once.

After each epoch, validation is performed and the result is compared with earlier epochs.

7. Checkpointing

FoliaScan saves:

```
best_model.pt
last_model.pt
```

- best_model.pt stores the epoch with the lowest validation loss.
- last_model.pt stores the most recently completed epoch.

The final epoch is not necessarily the best epoch.

A checkpoint also records information such as:

- Model state
- Optimiser state
- Class mapping
- Training configuration
- Random seed
- Epoch and metrics

8. Early stopping

Early stopping ends training when validation loss does not improve for a configured number of epochs.

It saves compute and prevents training from continuing without useful improvement.

9. Training history

The training process writes:

```
history.csv
history.json
```

These files record training and validation metrics for each epoch.

They can later support learning curves, experiment comparison, and MLflow tracking.

10. Baseline model

A baseline is a dependable reference model, not necessarily the best possible model.

It helps answer:

- Does training work end to end?
- What can a simple model achieve?
- Do later changes improve the result?
- Can the same workflow run locally and in Azure?

Important note

Validation selects the checkpoint. The test set must remain untouched until model development is complete.

Results from a one-epoch integration check prove that the pipeline works; they should not be reported as final model performance.

Essential command

```
poetry run python -m foliascan.training.train `
  --manifest data/processed/dataset_manifest.csv `
  --data-dir data/raw/plantvillage_tomato_color `
  --config configs/training.example.yaml `
  --epochs 10 `
  --device cuda `
  --output-dir artifacts/training/resnet18_baseline_v1
```

Review questions

1. What happens during the forward pass?
2. What is the difference between backpropagation and the optimiser?
3. Why are gradients disabled during validation?
4. Why can best_model.pt be better than last_model.pt?
5. What problem does early stopping address?
6. Why is a baseline needed before trying more complex models?

Key takeaway

A reliable training workflow must manage loss, gradients, optimisation, validation, checkpointing, outputs, and reproducibility—not only run the model for several epochs.

- 1.6 Evaluation & error analysis

Objective

Evaluate the selected checkpoint on the untouched test split and investigate the model's main failure patterns.

What was implemented

The evaluator:

- Loads best_model.pt.
- Restores the model configuration and class mapping.
- Loads only test records.
- Uses deterministic transforms.
- Disables gradient calculation.
- Calculates aggregate and per-class metrics.
- Creates a confusion matrix.
- Saves predictions and misclassification reports.

Concepts I should understand

1. Evaluation mode

During evaluation:

```
model.eval()
```

switches layers such as dropout and batch normalisation to evaluation behaviour.

```
torch.inference_mode()
```

disables gradient tracking because model parameters are not being updated.

This reduces memory use and unnecessary computation.

2. Accuracy

Accuracy is the proportion of all predictions that are correct.

It is useful as a general summary but does not show which classes are difficult.

3. Precision

Precision asks:

When the model predicted this class, how often was it correct?

High precision means few false positives.

4. Recall

Recall asks:

Of all real examples of this class, how many did the model find?

High recall means few missed examples.

5. F1-score

F1 combines precision and recall.

It is useful when both false positives and false negatives matter.

6. Macro and weighted metrics

Macro metrics give every class equal importance.

Weighted metrics give more influence to classes containing more samples.

Macro F1 is especially useful when class sizes are unequal and performance on smaller classes matters.

7. Confusion matrix

A confusion matrix shows which classes are confused with each other.

For example:

```
True class:      Early blight
Predicted class: Late blight
```

Repeated confusion between the same classes may indicate visual similarity or insufficient model generalisation.

8. Misclassification analysis

The evaluator records information such as:

- Image path
- True class
- Predicted class
- Confidence
- Correctness

High-confidence wrong predictions are particularly useful for finding:

- Incorrect labels
- Similar classes
- Background shortcuts
- Unusual images
- Weak generalisation

9. Domain shift

PlantVillage images were collected under relatively controlled conditions.

Real field images may have:

- Complex backgrounds
- Different lighting
- Occluded leaves
- Different cameras
- Weather effects
- Multiple diseases

Strong test results on PlantVillage therefore do not automatically guarantee strong real-world performance.

Important note

The test results must not be used repeatedly to tune the model. Once test results influence model changes, the test set is no longer an independent final evaluation.

Essential command

```
poetry run python -m foliascan.evaluation.evaluate `
  --manifest data/processed/dataset_manifest.csv `
  --data-dir data/raw/plantvillage_tomato_color `
  --checkpoint artifacts/training/resnet18_baseline_v1/best_model.pt `
  --output-dir artifacts/evaluation/resnet18_baseline_v1 `
  --device cuda
```

Review questions

1. Why are gradients disabled during evaluation?
2. What is the difference between precision and recall?
3. Why can macro F1 be more informative than accuracy?
4. What information does a confusion matrix provide?
5. Why should high-confidence wrong predictions be inspected?
6. Why does strong PlantVillage performance not guarantee field performance?

Key takeaway

Evaluation should explain not only how often the model is correct, but also which classes fail, how they fail, and whether the test data represents the intended real-world environment.

Phase 2 — Create Azure infrastructure

Goal

The goal of Phase 2 is to create a secure, reproducible, and cost-conscious Azure Machine Learning environment for FoliaScan.

The local ML workflow already exists. This phase prepares the cloud resources required to store assets, run jobs, manage models, and support later deployment.

Complete workflow

```
Verify the Azure account and subscription
        ↓
Create and organise the resource group
        ↓
Create the Azure ML workspace
        ↓
Provision CPU and GPU compute
        ↓
Review identities, permissions, and costs
        ↓
Connect the local project to Azure
```

Main concepts

1. Azure resource hierarchy

Azure resources are organised through this hierarchy:

```
Azure account
      ↓
Subscription
      ↓
Resource group
      ↓
Individual resources
```

- A **subscription** is the billing, quota, and access boundary.
- A **resource group** organises related resources.
- Resources include the Azure ML workspace, storage, compute, and supporting services.

2. Azure Machine Learning workspace

The workspace is the central management area for an Azure ML project.

It connects:

- Data assets
- Environments
- Jobs and experiments
- Compute resources
- Models
- Endpoints
- Logs and monitoring

The workspace manages these components but is not itself the machine that trains the model.

3. Reproducible infrastructure

Cloud resources should be created from reviewable configurations and commands rather than only through manual portal clicks.

FoliaScan uses:

- Azure CLI
- Azure ML CLI v2
- YAML configuration files

This makes infrastructure easier to reproduce, inspect, and later automate.

4. Compute and cost control

Cloud compute is paid infrastructure. The project therefore separates CPU and GPU workloads and uses scale-to-zero settings to avoid paying for inactive machines.

5. Identity and access

Cloud resources must know:

- Who or what is connecting
- How that identity authenticates
- What it is authorised to do
- Where the permission applies

Credentials should not be stored directly in source code.

Key takeaway

Phase 2 creates the managed cloud foundation around the local ML workflow. A correct Azure setup requires more than creating a VM: subscription, region, workspace, compute, identity,

permissions, and cost controls must work together.

- 2.1 Azure readiness and subscription setup

Objective

Verify that the local computer and Azure account are ready before creating cloud resources.

What was implemented

The setup confirmed that:

- Azure CLI was installed.
- The account could authenticate.
- The correct subscription was active.
- The Azure ML CLI extension was available.
- Required resource providers were registered.
- Azure ML-supported regions could be inspected.

Concepts I should understand

1. Azure subscription

A subscription controls:

- Billing
- Available quotas
- Regions and services
- User permissions
- Spending limits

Azure CLI performs operations in one active subscription at a time.

Selecting the wrong subscription can cause unexpected billing, missing permissions, or resources being created in the wrong environment.

2. Azure CLI and Azure ML extension

Azure CLI is the general command-line interface:

```
az
```

The Azure ML extension adds machine-learning commands:

```
az ml
```

The relationship is:

```
Azure CLI
└── Azure ML extension
    └── workspace, compute, data, job, model and endpoint commands
```

3. Resource provider

A resource provider enables Azure to create and manage a particular resource type.

Examples:

```
Microsoft.MachineLearningServices → Azure Machine Learning
Microsoft.Storage                 → Storage accounts
Microsoft.KeyVault                → Key Vault
```

A provider may need to be registered before its resources can be created.

4. Azure region

A region is the geographical location where Azure resources are hosted.

Region choice affects:

- Service availability
- VM and GPU availability
- Quota
- Cost
- Latency
- Data-location requirements

The nearest region is not automatically the best region for every workload.

Important note

Always verify the active subscription before creating paid resources. Subscription IDs, tenant IDs, and credentials should not appear in public screenshots or documentation.

Essential commands

```
az version
az login
az account show --output tableaz extension add --name ml
az ml --helpaz provider show `
  --namespace Microsoft.MachineLearningServices `
  --query "{Provider:namespace, State:registrationState}" `
  --output table
```

Review questions

1. What is the role of an Azure subscription?
2. What could happen if the wrong subscription is active?
3. What is the relationship between az and az ml?
4. Why must resource-provider registration be checked?
5. Why can region choice affect an ML workload?

Key takeaway

Before creating infrastructure, confirm the subscription, authentication, CLI tools, providers, permissions, and region. Many Azure failures come from configuration rather than ML code.

- 2.2 Resource naming and resource group

Objective

Create a dedicated resource group and establish a consistent naming convention for FoliaScan resources.

What was implemented

The following decisions were made:

```
Region:         Norway East
Resource group: rg-foliascan-dev-ne
Workspace:      mlw-foliascan-dev-ne
Environment:    Development
```

Naming pattern:

```
<resource-type>-<project>-<environment>-<region>
```

Example:

```
rg-foliascan-dev-ne
```

Concepts I should understand

1. Resource group

A resource group is a logical container for related Azure resources.

For FoliaScan, it can contain:

```
rg-foliascan-dev-ne
├── Azure ML workspace
├── Storage account
├── Key Vault
├── Container Registry
└── Compute resources
```

It supports organisation, access control, cost inspection, and cleanup.

2. Naming convention

A consistent name should make the resource's purpose understandable.

Useful parts include:

- Resource type
- Project
- Environment
- Region

Clear names become especially important when a company manages many projects and environments.

3. Environment separation

Resources from different lifecycle stages should be separated:

```
dev  → development
test → testing
prod → production
```

Development changes should not accidentally affect production resources or costs.

4. Tags

Tags are key-value metadata attached to resources.

Examples:

```
project=foliascan
environment=dev
purpose=learning
```

They help with:

- Cost tracking
- Filtering
- Ownership
- Governance

Important note

Deleting a resource group may delete all resources inside it. Always inspect its contents first.

Essential commands

```
az group create `
  --name rg-foliascan-dev-ne `
  --location norwayeast `
```

```
  --tags project=foliascan environment=dev purpose=learningaz group show `
  --name rg-foliascan-dev-ne `
  --output tableaz resource list `
  --resource-group rg-foliascan-dev-ne `
  --output table
```

Review questions

1. What is the purpose of a resource group?
2. Why should development and production resources be separated?
3. How does a naming convention improve cloud management?
4. Why are tags useful?
5. What is the risk of deleting a resource group?

Key takeaway

Cloud resources should be organised before they are created. Resource groups, clear names, environments, and tags make the infrastructure easier to understand, manage, and clean up.

- 2.3 Azure Machine Learning workspace

Objective

Create the central Azure Machine Learning workspace using a version-controlled YAML configuration.

What was implemented

A development workspace was created:

```
Workspace:      mlw-foliascan-dev-ne
Resource group: rg-foliascan-dev-ne
Region:         Norway East
```

It was defined in:

```
infra/azure/workspace.yml
```

Concepts I should understand

1. Azure ML workspace

The workspace manages and connects:

- Data assets
- Environments
- Compute
- Jobs and experiments
- Models
- Endpoints
- Logs

It is a management hub, not a training machine.

Training happens on a compute target connected to the workspace.

2. Supporting resources

An Azure ML workspace uses supporting Azure services.

Storage account

Stores data, job outputs, model artifacts, and logs.

Key Vault

Stores secrets and sensitive configuration securely.

Container Registry

Stores container images used for training and deployment environments.

Application Insights

Collects monitoring, request, performance, and error information.

These services have different responsibilities even though the workspace connects them together.

3. Declarative YAML configuration

A YAML file describes the desired workspace configuration.

Benefits compared with portal-only creation:

- Version controlled
- Reviewable
- Repeatable
- Easier to automate
- Less dependent on manual steps

4. Provisioning state

Provisioning is the process of creating or updating a cloud resource.

A state such as:

```
Succeeded
```

means Azure completed the requested operation.

A failed state requires inspection of the region, permissions, configuration, quota, or service availability.

Important note

Creating the workspace may also create supporting resources. Some of them can produce small costs even when no GPU job is running.

Essential commands

```
az ml workspace create `
  --file infra/azure/workspace.ymlaz ml workspace show `
  --name mlw-foliascan-dev-neaz configure --defaults `
  group=rg-foliascan-dev-ne `
  workspace=mlw-foliascan-dev-ne `
  location=norwayeast
```

Review questions

1. Why is the workspace not the same as compute?
2. What does the workspace manage?
3. What roles do Storage, Key Vault, Container Registry, and Application Insights play?
4. Why is YAML preferable to portal-only configuration?
5. What does provisioning state describe?

Key takeaway

The workspace is the central control and management layer of Azure Machine Learning. It connects data, software environments, compute, jobs, models, and deployment resources.

- 2.4 CPU and GPU compute

Objective

Create managed compute resources for preprocessing, testing, and model-training jobs.

What was implemented

A managed CPU cluster was created:

```
Name:            cpu-fs-dev
VM size:         Standard_DS3_v2
Minimum nodes:   0
Maximum nodes:   1
Idle scale-down: 120 seconds
Tier:            Dedicated
```

GPU availability and quota were investigated separately. The notes record an initial regional VM-size failure, while the later connection test confirms that both cpu-fs-dev and gpu-fs-dev were discovered with a Succeeded state.

Concepts I should understand

1. Compute target

A compute target is the resource on which Azure ML executes work.

Examples include:

- Compute cluster
- Compute instance
- Kubernetes
- Other connected platforms

2. Compute cluster versus compute instance

Compute instance**Compute cluster**Interactive development machine | Job-oriented compute
Often used by one person | Used for submitted workloads
Suitable for notebooks and debugging | Suitable for repeatable training
May stay running | Can scale down automatically

FoliaScan uses clusters because its main purpose is running submitted jobs rather than maintaining a personal cloud workstation.

3. CPU and GPU workloads

CPU compute is suitable for:

- Validation
- Data preparation
- Small smoke tests
- Lightweight scripts

GPU compute is suitable for:

- Neural-network training
- Large tensor operations
- Compute-intensive deep learning

Using separate clusters avoids paying GPU prices for tasks that do not need a GPU.

4. VM size and VM family

A VM size defines the hardware of one node:

- CPU cores
- Memory
- GPU type and count
- GPU memory

Related VM sizes belong to a VM family.

Azure quotas are often assigned to the family rather than only to one VM size.

5. Availability and quota

These are separate checks:

- **Availability:** Does Azure offer this VM size in the region?
- **Quota:** Is this subscription allowed to use enough cores from its family?

A VM can appear in a list but still fail because of:

- Zero family quota
- Insufficient available quota
- Unsupported region
- Temporary Azure capacity limits

6. Autoscaling and scale to zero

FoliaScan uses:

```
min_instances: 0
max_instances: 1
```

- Minimum zero prevents paying for idle nodes.
- Maximum one avoids accidental multi-node costs and stays within quota.

7. Dedicated and low-priority compute

- **Dedicated compute:** more reliable, but more expensive.
- **Low-priority compute:** cheaper spare capacity that Azure may interrupt.

Low-priority machines are more suitable for retryable experiments than for the first complete cloud workflow.

Important note

Disabling public SSH access and disabling public IP addresses are different security settings. No public SSH prevents direct login, but the node may still have a public IP unless that is disabled separately.

Essential commands

```
az ml compute listaz ml compute list-sizes --location norwayeast
az ml compute list-usage --location norwayeastaz ml compute create `
  --file infra/azure/compute/cpu-cluster.yml
```

Review questions

1. What is the difference between a compute cluster and a compute instance?
2. Why should CPU and GPU tasks use separate compute?
3. What is the difference between VM size and VM family?
4. Why can VM creation fail even when the size appears available?
5. Why is min_instances: 0 useful?
6. When is low-priority compute appropriate?
7. What caused the original GPU provisioning problem?

Key takeaway

Compute selection requires balancing workload requirements, regional availability, quota, reliability, security, and cost—not simply choosing the most powerful machine.

- 2.5 Identity, permissions, and cost controls

Objective

Understand who can access the Azure resources, what they are allowed to do, and how unnecessary spending is reduced.

What was implemented

- User access and inherited roles were inspected.
- The learning subscription's Owner access was identified.
- The workspace identity was reviewed.
- No unnecessary role assignments were added.
- Both clusters were configured to scale from zero to one node.
- Paid jobs were avoided without confirmed free credits.

Concepts I should understand

1. Identity

An identity can be:

- A user
- An application
- An Azure resource

It represents who or what is requesting access.

2. Authentication and authorization

- **Authentication:** verifies who the identity is.
- **Authorization:** determines what that identity may do.

Successful login does not automatically mean permission to perform every operation.

3. Azure RBAC

Azure Role-Based Access Control combines:

```
identity + role + scope
```

Example:

```
User + Contributor role + FoliaScan resource group
```

4. Scope

Scope defines where the permission applies:

```
Subscription
    ↓
Resource group
    ↓
Individual resource
```

A smaller scope is safer when wider access is unnecessary.

5. Managed identity

A managed identity allows an Azure resource to authenticate to another Azure service without storing a password or secret in the source code.

6. Least privilege

An identity should receive only the permissions it needs.

Owner access is practical for a personal learning subscription but is normally too broad for an employee working in a company environment.

7. Cost controls

Important controls include:

- Scale to zero
- Maximum node limits
- Short idle time
- Choosing suitable VM sizes
- Checking active resources
- Budget alerts

A budget alert sends a warning. It does not automatically stop resources or guarantee zero cost.

Important note

Autoscaling controls compute-node cost, but the workspace and supporting services may still produce charges. Scale-to-zero does not mean the complete Azure environment is free.

Essential commands

```
az ad signed-in-user showaz role assignment list `
  --assignee <object-id>az ml workspace show --query identity
az ml compute list
```

Review questions

1. What is the difference between authentication and authorization?
2. What three elements form an RBAC assignment?
3. Why are managed identities safer than credentials in code?
4. What does permission scope mean?
5. Why is least privilege important?
6. Why does scale-to-zero not guarantee zero Azure cost?
7. Why is a budget alert not a hard spending limit?

Key takeaway

Responsible cloud engineering requires secure identities, limited permissions, appropriate scopes, and active cost controls. Access and spending should never be left to default assumptions.

- 2.6 Local-to-Azure connection test

Objective

Confirm that the local FoliaScan Python environment can authenticate and connect to the real Azure ML workspace without submitting jobs or changing resources.

What was implemented

A read-only connection test was implemented using Azure ML SDK v2.

It confirmed:

```
Connection: connected
Workspace:  mlw-foliascan-dev-ne
Region:     Norway East

cpu-fs-dev → AmlCompute → Succeeded
gpu-fs-dev → AmlCompute → Succeeded
```

The test did not:

- Upload data
- Start compute
- Submit training
- Create resources
- Modify existing resources

Concepts I should understand

1. Azure ML SDK v2

The SDK allows Python code to work programmatically with:

- Workspaces
- Compute
- Data
- Environments
- Jobs
- Models
- Endpoints

Azure CLI is useful for shell commands and automation. The SDK is useful when Azure operations are part of a Python application.

2. MLClient

MLClient is the main Python client for connecting to a specific Azure ML workspace.

It needs the:

- Subscription
- Resource group
- Workspace name
- Authentication credential

After connecting, it can inspect or manage Azure ML resources.

3. DefaultAzureCredential

DefaultAzureCredential tries supported authentication methods without hardcoding credentials.

During local development, it can reuse the existing Azure CLI login created with:

```
az login
```

4. Environment variables

Values such as the subscription ID, resource group, and workspace name should be kept outside the application code.

Example variable names:

```
AZURE_SUBSCRIPTION_ID
AZURE_RESOURCE_GROUP
AZURE_ML_WORKSPACE
```

Real secret values should not be committed.

5. Read-only operation

A read-only test retrieves existing information without creating, updating, or deleting resources.

Listing compute proves that authentication, permissions, workspace configuration, and

network access work together.

6. Real integration test versus mock

A mocked test checks application logic without contacting Azure.

A real read-only test confirms that the actual Azure account and workspace can be reached.

Both are useful:

- Mocks support fast automated tests.
- A real read confirms actual cloud connectivity.

Important note

Discovering a compute cluster does not start it. Submitting a job is a separate action that may activate nodes and generate costs.

Essential commands

```
az loginpoetry run python -m foliascan.cloud.azure_connection
```

Review questions

1. What is the role of MLClient?
2. How can DefaultAzureCredential authenticate locally?
3. Why should Azure identifiers and credentials not be hardcoded?
4. Why is a real workspace read useful?
5. What is the difference between listing compute and submitting a job?
6. Why are mocked integration tests still necessary?

Key takeaway

The connection test proves that the local project can securely reach and inspect the real Azure ML environment while keeping configuration outside the code and avoiding paid or destructive operations.

Phase 3 — Move and Version the Dataset in Azure

Goal

The goal of Phase 3 is to move the verified local dataset into Azure and register it as reusable, versioned Azure Machine Learning data assets.

This allows future training jobs to reference stable asset names and versions instead of depending on local folders or manually entered storage paths.

Complete workflow

```
Inspect the workspace storage and datastores
        ↓
Define the required Azure data assets
        ↓
Describe the assets in version-controlled YAML
        ↓
Upload and register the assets
        ↓
Verify their metadata and local data integrity
```

Main concepts

1. Storage account, datastore, and data asset

These three objects have different responsibilities:

```
Storage account → physically stores files
Datastore       → connects Azure ML to storage
Data asset      → named and versioned reference to data
```

A training job normally uses a data asset without needing to know the full physical storage path or credentials.

2. Data versioning

A data asset version represents one specific state of the data.

```
foliascan-tomato-images:1
foliascan-tomato-images:2
```

When the content or preparation changes, a new version should normally be registered instead of silently replacing the existing version.

3. Data lineage

Data lineage records which data version was used by a job, experiment, or model.

This makes it possible to answer:

Which exact dataset produced this model?

4. Separation of data and metadata

FoliaScan keeps three separate assets:

- Tomato images
- Final leakage-safe dataset manifest
- Original source manifest

This separates the physical images, FoliaScan's split decisions, and the original PlantVillage metadata.

5. Verification before training

Successful registration does not automatically prove that everything is correct.

Before training, the project should verify:

- Asset names and versions
- Asset types
- Required metadata and tags
- Local image and manifest counts
- Integrity fingerprints

Key takeaway

Phase 3 turns local files into identifiable, versioned, reusable ML inputs. This is an important MLOps step because cloud training should depend on explicit dataset versions rather than changing local folders.

- 3.1 Inspect storage and the default datastore

Objective

Understand the existing Azure ML storage environment before uploading or registering the FoliaScan dataset.

What was implemented

The workspace contained four managed datastores:

```
workspaceblobstore
workspaceartifactstore
workspacefilestore
workspaceworkingdirectory
```

workspaceblobstore was selected for the FoliaScan image and manifest assets.

The local dataset was also measured:

```
Images: 18,160
Approximate size: 0.52 GB
```

No new storage account or datastore was required, and no data or compute node was activated during this inspection.

Concepts I should understand

1. Storage account

A storage account is the Azure resource that physically stores files and objects.

The Azure ML workspace uses storage for items such as:

- Training data
- Job outputs
- Model artifacts
- Logs
- Temporary files

2. Datastore

A datastore is an Azure ML connection to a storage location.

It allows Azure ML jobs to access data without hardcoding storage credentials into training code.

A datastore is therefore a **connection**, not the dataset itself.

3. Default datastore

The default datastore is the storage location Azure ML normally uses when a command uploads local files without specifying another destination.

For FoliaScan:

```
Default Blob datastore → workspaceblobstore
```

4. Storage and compute are separate

Listing datastores or inspecting data assets is a control-plane operation.

It does not:

- Start CPU or GPU nodes
- Run training
- Load the dataset into memory

Important note

The existing workspace-managed storage was sufficient. Creating an additional storage account without a clear reason would add complexity and possibly extra cost.

Essential commands

```
az ml datastore listaz ml datastore show `
  --name workspaceblobstoreaz ml data list
```

Review questions

1. What is the difference between a storage account and a datastore?
2. What is the purpose of the default datastore?
3. Why was workspaceblobstore suitable for FoliaScan?
4. Why should dataset size be checked before uploading?
5. Does listing datastores start an Azure compute cluster?

Key takeaway

Inspect existing storage before creating anything new. The workspace already had a suitable Blob datastore for the image folder and manifests.

- 3.2 Define the Azure data-asset structure

Objective

Decide which reusable Azure ML data assets the project needs and how they should be separated.

What was implemented

Three data assets were defined:

```
foliascan-tomato-images
→ uri_folder

foliascan-dataset-manifest
→ uri_file

foliascan-source-manifest
→ uri_file
```

Each asset was designed with:

- A clear name
- An explicit version
- An asset type
- A source path
- A description
- Project and content tags

Concepts I should understand

1. Azure ML data asset

A data asset is a named and versioned reference that Azure ML jobs can use as an input.

Instead of using:

```
Long storage path
```

a training job can reference:

```
foliascan-tomato-images:1
```

2. uri_folder

uri_folder represents a directory containing multiple files.

It is appropriate for:

```
18,160 tomato images
```

Azure ML can mount or download the complete folder for a job.

3. uri_file

uri_file represents one individual file.

It is appropriate for:

```
dataset_manifest.csv
plantvillage_source_manifest.csv
```

4. Why the assets are separate

The three assets represent different information:

AssetResponsibilityTomato images | Physical image data
Dataset manifest | Final train, validation, and test decisions
Source manifest | Original PlantVillage metadata

Separating them improves auditability and allows one component to change without treating the entire dataset as one unclear object.

5. Asset version

Version 1 represents the current verified state.

A meaningful change should normally create Version 2, for example:

- New images
- Corrected labels
- New split logic
- Updated manifest rows

Important note

Do not overwrite Version 1 simply because the local files changed. A silent replacement would weaken reproducibility and data lineage.

Review questions

1. What is an Azure ML data asset?
2. What is the difference between uri_folder and uri_file?
3. Why were the images and manifests registered separately?
4. What should happen when the dataset content changes?
5. Why is a friendly asset name better than using a long storage path directly?

Key takeaway

Design the data-asset structure before uploading. Clear asset boundaries and explicit versions make future training inputs easier to understand and reproduce.

- 3.3 Prepare version-controlled data YAML files

Objective

Describe each Azure ML data asset in a declarative YAML file that can be reviewed and stored in Git.

What was implemented

Three YAML files were created:

```
infra/azure/data/
├── tomato-images.yml
├── dataset-manifest.yml
└── source-manifest.yml
```

The repository currently contains all three definitions.

Concepts I should understand

1. Declarative configuration

A declarative file describes the desired resource:

This data asset should have this name, version, type, path, description, and tags.

Azure ML then performs the required operations to create that result.

2. YAML asset fields

A simplified FoliaScan YAML definition is:

```yaml
$schema: https://azuremlschemas.azureedge.net/latest/data.schema.json

name: foliascan-tomato-images
version: 1
type: uri_folder

path: ../../../data/raw/plantvillage_tomato_color

description: PlantVillage Tomato image subset used by FoliaScan.

tags:
  project: foliascan
  source: plantvillage
  content: tomato-images
```

The actual tomato-images.yml uses this structure.

3. Schema

The $schema field tells editors and Azure ML which YAML structure is expected.

It can help identify:

- Unsupported properties
- Incorrect field names
- Invalid asset types
- Structural errors

4. Local path

The path field identifies the local file or folder to upload.

Examples:

```
Image folder → data/raw/plantvillage_tomato_color
Manifest     → data/processed/dataset_manifest.csv
```

The relative path is resolved from the YAML file's location.

5. Tags

Tags provide searchable metadata.

FoliaScan uses tags such as:

```
project: foliascan
source: plantvillage
content: tomato-images
```

Tags help distinguish assets when a workspace contains many datasets.

6. YAML and Git

The YAML definitions belong in Git because they are small configuration files describing the infrastructure.

The actual raw dataset does not belong in Git because it is large and managed separately.

Important note

Preparing a YAML file does not upload anything by itself. Upload and registration happen only when the Azure ML create command is executed in subphase 3.4.

Review questions

1. What does a declarative YAML file describe?
2. What is the purpose of the $schema field?
3. What information do name, version, type, and path provide?
4. Why should YAML files be committed while the raw images are excluded?
5. Why are tags useful?

Key takeaway

Version-controlled YAML makes the data-asset definitions reviewable and repeatable. It separates the description of an asset from the command that creates it.

- 3.4 Upload and register versioned data assets

Objective

Upload the local dataset to Azure Storage and register it as reusable Azure ML data assets.

What was implemented

Before uploading, the local inputs were checked:

```
Images:                 18,160
Image-folder size:      about 554 MB
Dataset-manifest rows:  18,160
Source-manifest rows:   18,160
```

Three Version 1 assets were uploaded to workspaceblobstore and registered:

```
foliascan-tomato-images:1
→ uri_folder

foliascan-dataset-manifest:1
→ uri_file

foliascan-source-manifest:1
→ uri_file
```

The two small manifest files were uploaded first to validate the YAML configuration and Azure permissions before transferring the larger image folder.

Concepts I should understand

1. Upload

Upload transfers the physical files from the local computer to Azure Storage.

```
Local files → workspaceblobstore
```

2. Registration

Registration creates a named and versioned Azure ML reference to the cloud location.

```
Cloud storage path
        ↓
foliascan-tomato-images:1
```

Upload and registration are related, but they are not the same concept.

3. Local-path behaviour

Because the YAML definitions contain local paths, Azure ML performs two operations:

```
Upload local content to the default datastore
        ↓
Register a data asset referencing the cloud content
```

4. AzureML URI

An AzureML URI is a workspace-aware path referencing data through Azure Machine Learning.

It can identify data using the workspace, datastore, path, asset name, and version instead of exposing storage credentials in the training code.

5. Data lineage in future jobs

When a training job uses:

```
foliascan-tomato-images:1
```

Azure ML can record that Version 1 was used.

If Version 2 is used later, the job history can distinguish the two datasets.

Important note

Uploading or registering data does not start the CPU or GPU clusters. However, cloud storage and data transfer are separate Azure resources and may still have costs.

Essential commands

```
az ml data create `
  --file infra/azure/data/dataset-manifest.ymlaz ml data create `
  --file infra/azure/data/source-manifest.ymlaz ml data create `
  --file infra/azure/data/tomato-images.ymlaz ml data list
```

Review questions

1. What is the difference between upload and registration?
2. What happens when az ml data create receives a local path?
3. Why were the manifests uploaded before the image folder?

4. What does an AzureML URI represent?
5. How does asset versioning support data lineage?
6. Does uploading a data asset start the training compute?

Key takeaway

Physical files are uploaded to storage, while registration creates the friendly, versioned Azure ML object that future jobs will use.

- 3.5 Verify data integrity and Azure access

Objective

Confirm that the registered assets are accessible and that their metadata matches the verified local dataset.

What was implemented

A read-only Python verification command confirmed:

```
Verification status:    verified
Registered assets:      3
Local images:           18,160
Dataset-manifest rows:  18,160
Source-manifest rows:   18,160
```

For each Azure data asset, the tool checked:

- Correct name
- Correct Version 1
- Correct asset type
- Required tags
- Presence of a cloud storage path

It also calculated SHA-256 fingerprints for the two local manifest files.

The verification did not upload, modify, download, or delete data and did not start a compute cluster.

Concepts I should understand

1. Data integrity

Data integrity means confidence that data is complete, consistent, and has not changed unexpectedly.

Useful checks include:

- File counts
- Manifest row counts
- Metadata
- Content hashes

2. SHA-256 fingerprint

SHA-256 calculates a fingerprint from file content.

```
File content
     ↓
SHA-256
     ↓
Fixed-length fingerprint
```

Changing even a small part of the file normally produces a different fingerprint.

A hash is stronger evidence of unchanged content than checking only the filename or file size.

3. Metadata verification

Metadata verification confirms that Azure registered the expected object:

```
Expected name
Expected version
Expected type
Expected tags
Cloud path exists
```

It checks the asset definition, not the model or training data behaviour.

4. Read-only verification

A read-only command retrieves and validates information without changing cloud resources.

This makes it suitable for checking access and configuration without producing new assets or compute costs.

5. Verification limitation

The verifier checks:

- Azure asset metadata
- Local image and manifest counts
- Local manifest hashes

It does **not** download every Azure image and compare every cloud byte with the local

dataset.

Therefore:

```
Metadata and local integrity verified
≠
Full byte-for-byte cloud dataset verification
```

This limitation is important to understand. The repository documentation explicitly describes the command as read-only and lists the exact metadata and local checks it performs.

Important note

Registration should always be followed by verification. A successful create command alone does not prove that the correct asset name, type, version, tags, and local dataset counts were used.

Essential command

```
poetry run python -m foliascan.cloud.azure_data_assets `
  --image-root data/raw/plantvillage_tomato_color `
  --dataset-manifest data/processed/dataset_manifest.csv `
  --source-manifest data/processed/plantvillage_source_manifest.csv
```

Optional local checks:

```
Get-FileHash <file> -Algorithm SHA256Import-Csv <manifest.csv>
```

Review questions

1. What does data integrity mean?
2. Why is SHA-256 more useful than checking only the filename?
3. Which metadata fields were verified for each asset?
4. Why should registration be followed by verification?
5. What makes the verification command read-only?
6. What does the verifier not compare byte-for-byte?
7. Why should Azure identifiers remain outside the source code?

Key takeaway

Cloud data should be verified before it is used for training. Verification confirms that the expected assets exist and that their metadata and local source files match the intended dataset state.

Phase 4 — Train the model in Azure

Goal

The goal of Phase 4 is to run the existing FoliaScan training workflow as reproducible Azure Machine Learning command jobs.

Instead of redesigning the model, this phase connects the components already prepared:

```
Source code
      +
Versioned data assets
      +
Azure ML environment
      +
CPU or GPU compute
      ↓
Azure ML command job
      ↓
Metrics and training artifacts
```

Complete workflow

```
Select a reproducible training environment
        ↓
Make the existing training command cloud-compatible
        ↓
Define a small CPU smoke-test job
        ↓
Define the full GPU training job
        ↓
Submit and monitor both jobs
        ↓
Download and verify metrics and artifacts
```

Main concepts

1. Environment and compute are different

```
Environment → software required by the job
Compute     → hardware on which the job runs
```

The environment contains Python, PyTorch, CUDA, system libraries, and other dependencies.

The compute target provides CPU, memory, and possibly GPU resources.

2. Azure ML command job

A command job describes:

- Which code to run
- Which command to execute
- Which environment to use
- Which compute target to use
- Which data assets are inputs
- Where outputs should be stored

The job definition is stored in YAML so it can be reviewed and reproduced.

3. Mounted inputs and managed outputs

Azure ML mounts registered data assets into the job container.

```
Registered data asset
        ↓
Read-only mounted path
        ↓
Training program
```

The training program writes generated files to a writable Azure-managed output directory.

4. Cloud job lifecycle

A cloud training job normally passes through several stages:

```
Submit
  ↓
Upload code
  ↓
Queue
  ↓
Provision compute
  ↓
Prepare environment
  ↓
Mount data
  ↓
Run training
  ↓
Save outputs
  ↓
Complete
  ↓
Scale compute down
```

5. Metrics and artifacts

- **Metrics** describe training behaviour, such as loss and accuracy.
- **Artifacts** are generated files, such as checkpoints and training histories.

Both are needed to understand and reproduce a training run.

Key takeaway

Phase 4 proves that the local FoliaScan workflow can run on managed Azure infrastructure using versioned inputs, reproducible software, controlled compute, and Azure-managed outputs.

- 4.1 Select the Azure ML training environment

Objective

Choose a reproducible software environment that can run the FoliaScan PyTorch workflow on both CPU and GPU compute.

What was implemented

FoliaScan uses the Microsoft-curated environment:

```
AzureML-acpt-pytorch-2.8-cuda12.6:15
```

The same exact environment version is referenced by both the CPU and GPU job definitions.

Concepts I should understand

1. Azure ML environment

An Azure ML environment defines the software required by a job, including:

- Operating system
- Python runtime
- Python packages
- PyTorch
- CUDA libraries
- System dependencies

It helps the same code run consistently on Azure compute.

2. Curated environment

A curated environment is prepared and maintained by Microsoft.

Benefits include:

- Faster setup than building a new image
- Common ML libraries already installed
- Tested PyTorch and CUDA combinations
- Reduced container-maintenance work

3. Custom environment

A custom environment is needed when the curated environment does not contain the required packages or system configuration.

It can be defined using:

- A Conda specification
- A Docker image
- A Dockerfile and additional dependencies

For this phase, the curated environment was sufficient, so creating a custom one would add unnecessary complexity.

4. Container image

The environment runs as a container image on Azure compute.

The container packages the runtime so the training job does not depend on software installed manually on the compute node.

5. Environment version pinning

Using:

```
AzureML-acpt-pytorch-2.8-cuda12.6@latest
```

could resolve to a different environment in the future.

Using:

```
AzureML-acpt-pytorch-2.8-cuda12.6:15
```

pins one exact version and improves reproducibility.

Important note

The Azure environment is independent of the local Poetry environment. They may contain compatible packages, but Azure does not automatically install the local poetry.lock inside the curated container.

Review questions

1. What is the difference between an environment and a compute target?
2. What does an Azure ML environment contain?
3. What is the difference between curated and custom environments?
4. Why was a curated environment suitable for the first Azure run?
5. Why is an exact environment version safer than @latest?

Key takeaway

The environment controls the software runtime, while compute provides the hardware. Pinning the environment version makes future runs more reproducible.

- 4.2 Adapt the training entry point for cloud inputs

Objective

Make the existing local training command work inside Azure ML without creating a separate cloud-only training script.

What was implemented

The same entry point is used locally and in Azure:

```
python -m foliascan.training.train
```

It accepts:

```
--data-dir
--manifest
--config
--output-dir
```

The input paths can be:

- Local relative paths
- Local absolute paths
- Absolute Linux mount paths created by Azure ML

The output artifacts are written below the supplied --output-dir:

```
best_model.pt
last_model.pt
history.csv
history.json
```

The implementation also supports limited-batch options for cloud smoke tests:

```
--max-train-batches
--max-validation-batches
```

The training pipeline continues to use only the train and validation splits; the test split remains excluded.

Concepts I should understand

1. Portable training entry point

A portable entry point can run in different environments without changing its core training

logic.

```
Local execution
      ↓
Same training module
      ↑
Azure execution
```

This avoids maintaining two separate implementations that may gradually behave differently.

2. Path portability

Azure jobs run inside Linux containers and provide paths such as:

```
/mnt/azureml/...
```

The program must use the supplied paths directly rather than assume that all data exists relative to the local repository root.

3. Separation of code and configuration

The Python module contains the training logic.

The configuration file contains settings such as:

- Batch size
- Learning rate
- Model choice
- Early-stopping patience

The Azure job YAML decides how the training program is executed in the cloud.

4. Managed output directory

The training program should not choose an arbitrary local output path inside the container.

It writes to the Azure-provided output directory so Azure can persist the artifacts after the compute node stops.

5. One source of training logic

Reusing the existing entry point keeps the following in one implementation:

- Dataset loading
- Transforms
- Model construction
- Training loop
- Validation
- Checkpointing
- Artifact writing

The repository documentation explicitly states that a separate Azure training program was not created.

Important note

Cloud compatibility should not change dataset splitting or model-selection rules. Azure changes where the workflow runs, not how train, validation, and test data are used.

Review questions

1. Why is one portable training entry point preferable to separate local and Azure scripts?
2. Why must the program support absolute Linux paths?
3. What is the role of --output-dir?
4. Why are limited-batch arguments useful?
5. Why must the test split remain excluded from the training job?

Key takeaway

The local training workflow was made portable rather than rewritten. Azure supplies paths and infrastructure, while the core ML behaviour remains the same.

- 4.3 Define a CPU smoke-test job

Objective

Create a small and low-cost Azure job that validates the entire cloud training pipeline before using GPU compute.

What was implemented

The job is defined in:

```
infra/azure/jobs/cpu-smoke-test.yml
```

Main configuration:

Setting	Value
Display name	foliascan-cpu-smoke-test
Experiment	foliascan-training
Compute	cpu-fs-dev
Device	CPU
Epochs	1
Training batches	2
Validation batches	1
Timeout	1,800 seconds

Registered inputs:

```
foliascan-tomato-images:1
foliascan-dataset-manifest:1
```

Named output:

```
training_output
```

The YAML uses read-only data mounts and a writable managed output.

Concepts I should understand

1. Command-job YAML

The CPU YAML defines the complete execution contract:

```
Code
Command
Environment
Compute
Inputs
```

```
Outputs
Limits
Tags
```

2. Code snapshot

The job uses:

```
code: ../../..
```

Azure uploads the relevant repository content as a job code snapshot.

This allows the remote compute node to run the project code without directly reading the GitHub repository.

3. Read-only mount

The data inputs use:

```
mode: ro_mount
```

The job can read the registered images and manifest but cannot modify the original assets.

4. Writable output mount

The output uses:

```
mode: rw_mount
```

Azure provides a writable directory and persists its contents in workspace storage after the job completes.

5. Limited-batch smoke test

Only two training batches and one validation batch are processed.

The purpose is to detect integration failures such as:

- Incorrect paths
- Missing packages
- Invalid job configuration
- Data-mounting failures
- Compute problems
- Artifact-writing failures

It is not intended to measure model quality.

6. Timeout

The job timeout prevents a small validation job from running indefinitely because of a problem.

Important note

The CPU smoke-test loss and accuracy are not meaningful model-performance results because only a few batches were processed.

Review questions

1. What does a command-job YAML define?
2. What is a code snapshot?
3. What is the difference between ro_mount and rw_mount?
4. Why does the CPU job process only a few batches?
5. Which failures can a cloud smoke test detect?
6. Why should its accuracy not be reported as model performance?

Key takeaway

The CPU job validates the complete cloud path at low cost before the project commits to a longer GPU training run.

- 4.4 Define the GPU training job

Objective

Define the complete ResNet18 training job using Azure GPU compute and the full train and validation datasets.

What was implemented

The job is defined in:

`infra/azure/jobs/gpu-training.yml`

Main configuration:

Setting	Value
Display name	foliascan-gpu-training
Experiment	foliascan-training
Compute	gpu-fs-dev
Device	CUDA
Epochs	10
Batch size	32
Optimizer	AdamW
Learning rate	0.001
Weight decay	0.0001
Timeout	7,200 seconds

It uses the same:

- Source code
- Training entry point
- Environment version
- Registered image asset
- Registered manifest asset
- Named output

Unlike the smoke test, it does not limit the number of train or validation batches.

Concepts I should understand

1. Full training job

A full training job processes all configured train and validation batches for each epoch.

Its metrics can support checkpoint selection, unlike smoke-test metrics.

2. CUDA device

The job explicitly passes:

```
--device cuda
```

This ensures the training program fails clearly if the GPU runtime is unavailable instead of silently falling back to CPU.

3. CPU smoke test versus GPU training

CPU smoke testGPU full training Validates integration | Trains the real baseline
CPU compute | GPU compute
One epoch | Ten epochs
Limited batches | All batches
Low cost | Higher cost
Metrics not meaningful | Validation metrics useful

4. Same inputs, different workload

Both jobs use the same Version 1 data assets.

This means differences between the two jobs come mainly from execution scale and compute rather than different dataset versions.

5. Job tags

Tags such as the following make cloud jobs easier to identify:

```
project=foliascan
stage=gpu-full-training
data-version=1
epochs=10
```

Important note

The GPU job still performs training and validation only. It does not run final test-set evaluation, register the model, or deploy it.

Review questions

1. What is the main difference between the CPU and GPU jobs?
2. Why does the GPU command explicitly request CUDA?

3. Why should both jobs use the same data-asset versions?
4. Why are exact environment and dataset versions important?
5. What responsibilities remain outside this training job?

Key takeaway

The GPU YAML scales the already validated workflow to full training without changing its core data, model, or artifact logic.

- 4.5 Submit and monitor the training job

Objective

Submit the CPU and GPU command jobs, monitor their execution, inspect their logs, and confirm successful completion.

What was implemented

The execution order was:

```
Submit CPU smoke test
        ↓
Confirm CPU status = Completed
        ↓
Submit GPU training
        ↓
Stream GPU logs
        ↓
Confirm GPU status = Completed
```

Azure uploaded a code snapshot of approximately 1.65 MB during job submission.

Both jobs completed successfully:

Job | Job name | Status | Compute
CPU smoke test | heroic_pump_6kvy3nnvpt | Completed | cpu-fs-dev
GPU training | cyan_boot_v9vg1214b3 | Completed | gpu-fs-dev

The GPU job completed all ten epochs.

Concepts I should understand

1. Job submission

The YAML definition becomes an actual Azure ML job when submitted with:

```
az ml job create `
  --file infra/azure/jobs/cpu-smoke-test.yml
```

The returned job name is the unique identifier used for later operations.

2. Job status

A job can pass through states such as:

```
Queued
Preparing
Running
Completed
Failed
Canceled
```

The final status must be checked directly rather than inferred from the local terminal.

3. Log streaming

Live logs are viewed using:

```
az ml job stream --name <job-name>
```

Log streaming is only a connection to the running job.

If the local stream is interrupted, the Azure job may continue running.

4. Status checking

The reliable way to inspect the job is:

```
az ml job show `
  --name <job-name> `
  --query status `
  --output tsv
```

5. CPU gate before GPU

The GPU job was submitted only after the CPU smoke test reached Completed.

This prevents spending GPU resources on a workflow that has not passed basic integration checks.

6. Cold-start overhead

The first GPU epoch took much longer than later epochs.

The first epoch included additional work such as:

- Compute startup
- Environment preparation
- Initial data access
- Container startup
- Framework warm-up

Later epochs mainly reflected actual training time.

Essential commands

```
# Submit
az ml job create --file <job-yaml>

# Stream logs
az ml job stream --name <job-name>

# Check final status
az ml job show `
  --name <job-name> `
  --query status `
  --output tsv
```

Important note

Stopping or losing az ml job stream does not automatically cancel the remote job. Job status and cancellation are separate operations.

Review questions

1. What happens when an Azure ML job is submitted?
2. Why is the returned job name important?
3. What is the difference between log streaming and job execution?
4. Why must status be checked separately?
5. Why was the GPU job submitted only after the CPU job completed?
6. Why can the first epoch take much longer than later epochs?

Key takeaway

Cloud training is asynchronous. Submission, log streaming, status checking, and artifact retrieval are separate operations that use the job identifier.

- 4.6 Track metrics and artifacts

Objective

Retrieve the Azure job outputs, inspect training metrics, identify the best checkpoint, and verify that the required artifacts were generated.

What was implemented

The CPU and GPU job names were saved locally:

```
artifacts/azure/job-records/
├── cpu-job-name.txt
└── gpu-job-name.txt
```

The named outputs were downloaded into:

```
artifacts/
└── azure/
    └── jobs/
        └── <job-name>/
            └── named-outputs/
                └── training_output/
```

Both jobs produced:

```
best_model.pt
last_model.pt
history.csv
history.json
```

All four required GPU artifacts were verified. The two checkpoint files were approximately 128.1 MB each.

Concepts I should understand

1. Named output

The YAML defines:

```
training_output
```

A named output allows Azure and the user to identify and retrieve one logical group of job-generated files.

2. Artifact download

Artifacts were downloaded with:

```
az ml job download `
  --name <job-name> `
  --output-name training_output `
  --download-path artifacts/azure/jobs/<job-name>
```

Downloading an artifact creates a local copy; the Azure-stored version remains available.

3. Training history

history.csv contains one row of metrics for each epoch and is convenient for:

- Pandas
- Excel
- Plotting
- Manual review

history.json stores the history in a structured format suitable for:

- Scripts
- APIs
- Automation
- Further processing

4. Best and last checkpoints

- best_model.pt contains the epoch with the lowest validation loss.
- last_model.pt contains the final completed epoch.

For this Azure GPU run, both corresponded to epoch 10 because epoch 10 produced the best validation loss.

5. Azure GPU training result

The main results were:

Metric | Result
Completed epochs | 10
Best epoch | 10
Best validation loss | 0.212930
Best validation accuracy | 92.8964%
Final training accuracy | 94.1544%
Early stopped | No

Validation accuracy temporarily dropped during epoch 8 but recovered in epochs 9 and 10.

6. Validation metrics are not test metrics

The reported 92.8964% is validation accuracy.

It is used to understand training and select the checkpoint. It is not final test accuracy.

7. Artifact integrity

File presence, size, and SHA-256 hashes can be checked to confirm that artifacts were downloaded and did not change unexpectedly.

8. Metrics files versus MLflow

In Phase 4:

```
Training code
   ↓
history.csv / history.json
   ↓
Manual inspection
```

In Phase 5:

```
Training run
   ↓
MLflow logging
   ↓
Central experiment comparison
```

Phase 4 saves and reviews metrics, but does not yet provide full MLflow experiment tracking.

Important note

The validation accuracy from this run should not be presented as final model performance. Final performance must come from evaluation of the selected checkpoint on the untouched test split.

Review questions

1. What is a named Azure ML output?
2. What is the difference between metrics and artifacts?
3. What information is stored in history.csv and history.json?
4. What is the difference between best_model.pt and last_model.pt?
5. Why did both checkpoints correspond to epoch 10 in this run?

6. Why is validation accuracy not the same as test accuracy?
7. What remains to be added in Phase 5 with MLflow?

Key takeaway

A completed job is useful only when its metrics, checkpoints, and metadata can be retrieved and interpreted. Phase 4 produced a verified Azure-trained checkpoint, while central experiment tracking remains the next step.

Phase 5 — Track experiments with MLflow

- 5.1 Inspect the Azure MLflow tracking setup

Purpose

The goal was to understand how Azure Machine Learning integrates with MLflow before modifying the training code.

Work completed

The Azure ML workspace was confirmed as the MLflow tracking backend.

The existing experiment was:

```
foliascan-training
```

The previous CPU and GPU jobs from Phase 4 were already grouped under this experiment.

The local environment was also inspected, and MLflow was not installed before starting Phase 5.

Important concepts

An **MLflow experiment** groups related training runs.

An **MLflow run** represents one execution of the training code.

Azure ML automatically provides the tracking context for submitted command jobs. Therefore, the training code did not need:

```
mlflow.set_tracking_uri(...)
mlflow.set_experiment(...)
mlflow.start_run(...)
```

Hard-coding the workspace tracking URI was intentionally avoided.

What I learned

Azure ML jobs already have a run identity. MLflow can attach parameters, metrics, and artifacts to that existing run instead of creating a second unrelated run.

Interview explanation

I first inspected the Azure ML tracking configuration and confirmed that the workspace already acted as an MLflow backend. I reused the Azure job run context instead of hard-coding a tracking URI or creating duplicate runs.

- 5.2 Add MLflow dependencies and tracking support

Purpose

The goal was to add MLflow support to the project while keeping tracking optional for local training.

Dependencies added

```
mlflow: 2.16.2
azureml-mlflow: 1.62.0.post5
```

The installation updated:

```
pyproject.toml
poetry.lock
```

The dependency installation and existing test suite completed successfully.

Implementation

A new helper module was added:

```
src/foliascan/training/mlflow_tracking.py
```

A new CLI flag was added:

```
--enable-mlflow
```

MLflow tracking remains disabled when the flag is omitted.

This prevents normal local training from unexpectedly creating or updating tracking runs.

Design decisions

The implementation uses manual MLflow logging rather than autologging.

Manual logging provides control over:

- Parameter names
- Metric names
- Epoch steps
- Final summary metrics
- Artifact selection

- Avoiding duplicate checkpoint uploads

Validation

The completed implementation passed:

```
135 tests
Ruff
mypy
Poetry validation
CLI help validation
YAML parsing
git diff --check
```

What I learned

Experiment tracking should be explicit and testable. It should not silently change the behaviour of local development workflows.

Interview explanation

I added opt-in MLflow tracking to the existing training CLI. Tracking remains disabled by default, and Azure ML jobs enable it explicitly through a command-line flag.

- 5.3 Log parameters, epoch metrics, and artifacts

Purpose

The goal was to capture enough information to understand, reproduce, and compare training runs.

Parameters logged

The run records configuration values such as:

```
model_name
epochs
batch_size
learning_rate
optimizer_name
weight_decay
image_size
pretrained
freeze_backbone
random_seed
requested_device
max_train_batches
max_validation_batches
```

These values describe how the run was configured.

Per-epoch metrics logged

After every epoch, the following values are logged:

```
train_loss
train_accuracy
validation_loss
validation_accuracy
learning_rate
elapsed_seconds
```

The epoch number is used as the MLflow metric step:

```
step = epoch
```

This enables Azure ML Studio to display metric curves across epochs.

Final summary metrics logged

At the end of training:

```
completed_epochs
best_epoch
best_validation_loss
best_validation_accuracy
early_stopped
```

were recorded.

Lightweight artifacts logged

The MLflow artifact area contains:

```
config/
history/
```

This includes files such as:

```
training.example.yaml
history.csv
history.json
```

Checkpoints intentionally excluded

The following large files were not duplicated in MLflow:

```
best_model.pt
last_model.pt
```

They remain in the Azure ML named output:

```
training_output
```

This avoids uploading approximately 128 MB checkpoints twice.

What I learned

The three main MLflow tracking categories are:

Category**Purpose**Parameters | Describe the run configuration
Metrics | Describe model behaviour and performance
Artifacts | Preserve supporting files

MLflow tracking is different from model registration. Tracking describes an experiment; registration creates a managed model version for later deployment.

Interview explanation

I logged configuration parameters once, training and validation metrics after every epoch, final summary values, and lightweight artifacts. Large checkpoints remained in the Azure named output to avoid unnecessary duplication.

- 5.4 Update the Azure ML job definitions

Purpose

The goal was to enable MLflow tracking for cloud jobs without changing the existing training workflow.

Files updated

```
infra/azure/jobs/cpu-smoke-test.yml
infra/azure/jobs/gpu-training.yml
```

Both commands now include:

```
--enable-mlflow
```

For example, the CPU job still preserves:

```
Experiment: foliascan-training
Compute: cpu-fs-dev
Epochs: 1
Maximum training batches: 2
Maximum validation batches: 1
```

The registered data assets, managed output, pinned environment, compute target, and timeout remained unchanged.

Why this matters

MLflow tracking is enabled through the version-controlled YAML file rather than through manual portal configuration.

This makes the tracking behaviour:

- Repeatable
- Reviewable
- Consistent
- Stored in Git
- Easy to reproduce

What I learned

The job YAML acts as the cloud execution contract. It controls the code, environment, data, compute, outputs, experiment name, and tracking option.

Interview explanation

I enabled MLflow tracking directly in the version-controlled Azure ML command-job definitions while preserving the existing compute, data assets, environment version, outputs, and execution limits.

- 5.5 Run and inspect tracked experiments

Purpose

The goal was to confirm that the MLflow implementation worked in a real Azure ML job.

Tracked CPU run

```
Display name: foliascan-cpu-smoke-test
Job name: frank_rocket_07jzflxzjg
Experiment: foliascan-training
Compute: cpu-fs-dev
Status: Completed
```

Run configuration

```
Epochs: 1
Batch size: 32
Maximum training batches: 2
Maximum validation batches: 1
Device: CPU
Learning rate: 0.001
Optimizer: AdamW
```

Logged metrics

The Azure ML Metrics page displayed:

```
best_epoch
best_validation_accuracy
best_validation_loss
completed_epochs
early_stopped
elapsed_seconds
learning_rate
train_accuracy
train_loss
validation_accuracy
validation_loss
```

Result

```
Best epoch: 1
Train loss: 2.101177
Train accuracy: 26.56%
Validation loss: 17.98589
Validation accuracy: 0%
Completed epochs: 1
Early stopped: False
Elapsed training time: 9.15 seconds
```

The total Azure job duration was longer because it included compute provisioning,

environment preparation, code setup, and data mounting.

Verification completed

The Azure ML Studio interface confirmed:

- Parameters were recorded.
- Metrics were recorded.
- The metric step was associated with the epoch.
- Config and history artifacts were present.
- The job remained linked to its Git commit.
- Registered data assets were displayed as inputs.
- The environment version was displayed.
- No model was registered.

Important interpretation

The smoke-test metrics are not meaningful indicators of final model quality because only two training batches and one validation batch were processed.

The purpose was to validate experiment tracking, not to train an accurate model.

Interview explanation

I validated the MLflow integration with a real Azure CPU smoke-test job. Azure ML Studio showed the logged parameters, metrics, artifacts, Git metadata, registered data inputs, and pinned environment.

- 5.6 Compare runs and document the results

Purpose

The goal was to demonstrate how MLflow can compare multiple runs with different parameter values.

First run

```
Display name: foliascan-cpu-smoke-test
Epochs: 1
Maximum training batches: 2
Maximum validation batches: 1
```

Second run

```
Display name: foliascan-cpu-mlflow-comparison
Epochs: 2
Maximum training batches: 5
Maximum validation batches: 2
Status: Completed
```

Comparison

ItemFirst runSecond run Epochs | 1 | 2
Maximum training batches | 2 | 5
Maximum validation batches | 1 | 2
Best epoch | 1 | 2
Best validation loss | 17.98589 | 122.1309
Best validation accuracy | 0% | 0%
Completed epochs | 1 | 2
Early stopped | No | No

The second run displayed metric curves with:

```
Step 1
Step 2
```

Examples included:

```
elapsed_seconds
learning_rate
train_accuracy
train_loss
validation_accuracy
validation_loss
```

The learning rate remained constant:

```
0.001
```

Result interpretation

The comparison successfully demonstrated that MLflow can:

- Store multiple runs under one experiment
- Record different parameter values
- Preserve metric history by step
- Generate charts across epochs
- Support side-by-side run analysis

The second run did not produce better validation results. This is not concerning because both runs were intentionally tiny CPU integration tests using only a few batches.

The comparison was designed to test the tracking system, not to identify the best model.

For meaningful model comparison, future runs should use:

- The complete training dataset
- The complete validation dataset
- The same evaluation protocol
- Controlled changes to one or more hyperparameters

What I learned

A fair experiment comparison should distinguish between:

```
Pipeline validation
```

and:

```
Model-quality evaluation
```

Smoke tests are useful for validating technical integration but should not be used to make model-selection decisions.

Interview explanation

I submitted a second tracked CPU run with different epoch and batch limits. Azure ML displayed two-step metric curves and allowed the runs to be compared by parameters, metrics, runtime, and final summaries. I treated the results as pipeline validation rather than model-quality evaluation because both runs used only a few batches.

Phase 6 — Register the best model

- 6.1 Select the model candidate

Purpose

The goal was to choose the correct trained checkpoint for registration in Azure Machine Learning.

The selected model had to come from the full GPU training job, not from the small CPU smoke tests used for pipeline and MLflow validation.

Selected Candidate

```
Source job: cyan_boot_v9vg1214b3
Display name: foliascan-gpu-training
Source output: training_output
Checkpoint: best_model.pt
Architecture: ResNet18
Number of classes: 10
Best epoch: 10
Best validation loss: 0.212930
Best validation accuracy: 0.928964
```

Planned Azure ML model asset:

```
Name: foliascan-tomato-resnet18
Version: 1
Type: custom_model
```

Selection Criterion

The model was selected using:

```
Lowest validation loss
```

The best checkpoint was therefore best_model.pt, rather than last_model.pt.

Why validation loss was used

Validation loss measures how well the model generalizes to data not used for weight updates.

The checkpoint with the lowest validation loss was considered the most suitable candidate for deployment.

Why CPU Smoke-Test Models Were Not Selected

The Phase 5 CPU runs processed only a small number of batches.

They were created to verify:

- Azure job execution
- MLflow tracking
- Parameter logging
- Metric logging
- Artifact handling

They were not intended to produce deployable models.

Important Distinction

```
Validation accuracy: 92.90%
```

This value is not final test accuracy.

The test split was not evaluated again during model registration.

What I Learned

Model registration should begin with an explicit candidate-selection rule. A model should not be registered simply because it is the most recent checkpoint.

Interview Explanation

I selected the checkpoint from the full GPU training run using the lowest validation loss. I excluded the CPU smoke-test models because those runs were designed only to validate the infrastructure and MLflow integration.

- 6.2 Validate the checkpoint for inference

Purpose

The goal was to confirm that the selected checkpoint contained everything required to reconstruct the model later inside an inference service.

Checkpoint Validation

The local copy of best_model.pt was inspected.

The checkpoint contained the required fields:

```
epoch
model_name
model_state_dict
optimizer_state_dict
class_to_index
training_config
train_metrics
validation_metrics
best_validation_loss
random_seed
```

Model Reconstruction

The ResNet18 model was recreated using the information stored inside the checkpoint.

Validation result:

```
Model reconstruction: passed
Architecture: resnet18
Classes: 10
Missing state keys: 0
Unexpected state keys: 0
Training mode: False
```

Meaning of the Result

```
Missing state keys: 0
```

means every parameter expected by the reconstructed ResNet18 model was available in the checkpoint.

```
Unexpected state keys: 0
```

means the checkpoint did not contain incompatible or extra model parameters.

```
Training mode: False
```

confirms that:

```
model.eval()
```

was applied successfully.

This is necessary for deterministic inference behaviour in layers such as batch normalization and dropout.

Strict Weight Loading

The checkpoint was loaded with:

```
model.load_state_dict(
    checkpoint["model_state_dict"],
    strict=True,
)
```

Using strict=True prevents silently loading an incompatible checkpoint.

CPU Loading

The checkpoint was loaded using:

```
map_location="cpu"
```

This confirmed that the GPU-trained model could later be used for CPU inference.

Checkpoint Fingerprint

A SHA-256 fingerprint was calculated:

```
14e79f5c40d591bc32d83c6b80689c76613391cf12abce9c0a1b7e6a4e5582fb
```

The hash identifies the exact checkpoint selected for registration.

What I Learned

A model should be validated before registration. File existence alone is not enough.

The following should be checked:

- Required metadata exists
- Architecture can be reconstructed
- Weights load strictly
- Class mapping is available
- The model can enter inference mode
- The artifact has a reproducible fingerprint

Interview Explanation

Before registering the model, I validated the checkpoint structure, reconstructed ResNet18, loaded the state dictionary with strict validation, confirmed that there were no missing or unexpected keys, and verified that the model could run in evaluation mode on CPU.

- 6.3 Prepare a version-controlled model YAML

Purpose

The goal was to define the Azure ML model asset in a version-controlled YAML file rather than registering it manually through the Azure portal.

Model YAML File

The following file was created:

```
infra/azure/models/foliascan-tomato-resnet18.yml
```

Main fields:

```
name: foliascan-tomato-resnet18
version: 1
type: custom_model
```

The model path points directly to the output of the GPU training job:

```
azureml://jobs/cyan_boot_v9vg1214b3/outputs/training_output/paths/best_model.pt
```

The YAML records the model name, explicit Version 1, custom model type, source job-output path, validation metrics, data-asset versions, environment version, random seed, and checkpoint SHA-256 hash.

Why custom_model Was Used

The artifact is a raw PyTorch checkpoint:

```
best_model.pt
```

It is not an MLflow model package containing files such as:

```
MLmodel
conda.yaml
python_env.yaml
```

Therefore, it was registered as:

```
custom_model
```

Metadata and Tags

The YAML includes tags for:

```
Project
Task
Framework
Architecture
Source job
Source output
Source filename
Selection criterion
Best epoch
Best validation metrics
Dataset version
Manifest version
Environment version
Random seed
Checkpoint hash
```

Examples:

```
project: foliascan
framework: pytorch
architecture: resnet18
source-job: cyan_boot_v9vg1214b3
best-epoch: 10
dataset-asset: foliascan-tomato-images:1
environment-version: 15
evaluation-scope: validation-only
```

Versioning

The version was explicitly defined as:

```
Version 1
```

Future approved models can be registered as:

```
Version 2
Version 3
...
```

without replacing Version 1.

Git Ignore Issue

The repository originally contained this pattern:

```
models/
```

It also ignored the nested directory:

```
infra/azure/models/
```

As a result, Git initially did not detect the YAML file.

The rule was corrected to:

```
/models/
```

This means only the root-level local model directory is ignored, while Azure model definitions under infra/azure/models/ remain version-controlled.

What I Learned

Cloud resources should be defined as code whenever possible.

A version-controlled model YAML provides:

- Reviewable metadata
- Reproducible registration
- Clear source lineage
- Explicit versioning
- Easier collaboration
- Git history

I also learned that .gitignore patterns without a leading slash can match nested directories.

Interview Explanation

I created a version-controlled Azure ML model definition that referenced the best checkpoint directly from the training job output. I included model-selection metadata, data and environment versions, and the checkpoint hash for traceability.

- 6.4 Register the model asset

Purpose

The goal was to create a managed and versioned model asset inside the Azure ML workspace.

Registration Command

The model was registered using:

```
az ml model create `
    --file infra/azure/models/foliascan-tomato-resnet18.yml `
    --resource-group $RESOURCE_GROUP `
    --workspace-name $WORKSPACE
```

Registered Asset

```
Model name: foliascan-tomato-resnet18
Version: 1
Type: CUSTOM
```

Direct Registration from Job Output

The checkpoint was registered directly from:

```
azureml://jobs/cyan_boot_v9vg1214b3/outputs/training_output/paths/best_model.pt
```

Therefore, it was not necessary to upload the approximately 128 MB model again from the local computer.

Registration vs. Training

Model registration did not:

- Retrain the model
- Start CPU or GPU compute
- Change model weights
- Evaluate the model
- Create an endpoint
- Deploy the model

It only created a managed Azure ML asset that points to the selected artifact.

Registration vs. MLflow Tracking

MLflow tracking

Used for:

- Parameters
- Metrics
- Training history
- Experiment comparison
- Lightweight artifacts

Model registration

Used for:

- Creating a named model asset
- Assigning a version
- Preserving approved model artifacts
- Supporting later deployment

What I Learned

Experiment tracking and model registration are related but separate MLOps activities.

A tracked run is not automatically a deployment-ready registered model.

Interview Explanation

I registered the selected PyTorch checkpoint as Version 1 of a custom Azure ML model asset. The registration referenced the named output of the original GPU job, so the model did not need to be uploaded again from my machine.

- 6.5 Verify the registered version and lineage

Purpose

The goal was to confirm that the registered asset contained the correct version, type, source, description, and tags.

Verification Result

Azure ML Studio displayed:

```
Name: foliascan-tomato-resnet18
Version: 1
Type: CUSTOM
Created by: Pegah Salehi
Created by job: cyan_boot_v9vg1214b3
```

The model page also showed:

- Description
- Tags
- Asset ID
- Source training job
- Model version
- Model type

Verified Lineage

The registered model is connected to:

```
GPU training job
    ↓
training_output
    ↓
best_model.pt
    ↓
foliascan-tomato-resnet18:1
```

This lineage makes it possible to trace the model back to the job that produced it.

Verified Metadata

Important registered tags included:

```
architecture: resnet18
best-epoch: 10
selection-criterion: lowest-validation-loss
dataset-asset: foliascan-tomato-images:1
```

```
source-output: training_output
framework: pytorch
checkpoint-sha256: 14e79f5c...
```

Local Metadata Copy

The Azure model metadata was saved locally as:

```
artifacts/azure/models/
└── foliascan-tomato-resnet18/
    └── 1/
        └── model-metadata.json
```

The artifacts/ directory is ignored by Git.

This is appropriate because the metadata may contain:

- Subscription IDs
- Workspace resource IDs
- Datastore paths
- Internal Azure URIs

Azure CLI Warning

The CLI displayed a message about:

```
DeploymentTemplateReferenceSchema
```

being experimental.

This was only an informational warning from the Azure SDK. The metadata command still completed successfully.

What I Learned

A registered model should be verified after creation. Registration success alone is not enough.

The following should be inspected:

- Correct name
- Correct version
- Correct type
- Correct source artifact
- Source job lineage
- Tags

- Description
- Creation metadata

Interview Explanation

After registration, I verified the model in Azure ML Studio and through the CLI. I confirmed its Version 1 metadata, custom model type, tags, checkpoint hash, and lineage back to the GPU training job.

- 6.6 Document the model and prepare the deployment handoff

Purpose

The goal was to prepare all information needed by the next phase, where the registered model will be deployed to an online endpoint.

Deployment Handoff

```
Registered model: foliascan-tomato-resnet18:1
Model type: custom_model
Framework: PyTorch
Architecture: ResNet18
Classes: 10
Input image size: 224 × 224
Checkpoint file: best_model.pt
Source job: cyan_boot_v9vg1214b3
Source output: training_output
Environment family: acpt-pytorch-2.8-cuda12.6
Environment version: 15
Selection criterion: lowest validation loss
Best validation loss: 0.212930
Best validation accuracy: 92.8964%
Checkpoint SHA-256:
14e79f5c40d591bc32d83c6b80689c76613391cf12abce9c0a1b7e6a4e5582fb
```

Information Available Inside the Checkpoint

The future scoring script can retrieve:

```
model_name
model_state_dict
class_to_index
training_config
image_size
random_seed
validation metrics
```

This prevents manually duplicating:

- Class names
- Class indices
- Model architecture
- Image size

What Phase 7 Still Needs

Because this is a raw PyTorch custom_model, deployment still requires:

```
Inference predictor
Azure scoring script
init() function
```

```
run() function
Inference environment
Endpoint YAML
Deployment YAML
Request sample
Local inference tests
Azure endpoint invocation
Endpoint cleanup
```

Important Deployment Decision

CPU inference is sufficient for the first endpoint.

Training required GPU acceleration, but one-image ResNet18 inference does not necessarily require a continuously running GPU deployment.

What I Learned

A model-registration phase should end with a clear deployment contract. The deployment team or next project phase should know:

- Which model version to use
- How to reconstruct it
- What preprocessing is required
- Which class mapping to use
- What artifact contains the model
- What validation result justified its selection

Interview Explanation

I documented a deployment handoff containing the registered model version, architecture, input size, class count, source job, environment version, validation metrics, and artifact fingerprint. This provided the information required to build the inference service without duplicating model metadata manually.

Phase 7 — Deploy an online endpoint

Concept | What it means

Inference contract | Defines request and response structureScoring script | Connects Azure requests to the ML inference codeinit() | Loads the model when the container startsrun() | Processes individual requestsAZUREML_MODEL_DIR | Azure-provided location of the deployed modelInference environment | Reproducible runtime dependenciesEndpoint | Stable API addressDeployment | Model + code + environment + computeLocal deployment | Docker-based pre-cloud testingHealth probe | Checks that the inference container remains available/score | HTTP route used for inferencePOST /score 200 | Successful cloud predictionVM-family quota | Limits compute available to Azure ML deploymentsResource provider | Azure service namespace that must be registeredCleanup | Removes continuously billed deployment resources

I deployed a custom PyTorch ResNet18 model to an Azure ML Managed Online Endpoint using version-controlled model, environment, endpoint, and deployment definitions. I implemented and tested the scoring service locally with Docker, diagnosed container import and dependency issues, resolved Azure resource-provider and ML-specific quota constraints, successfully invoked the model through the cloud endpoint, inspected the deployment logs, and removed the endpoint afterward to control cost.

- 7.1 Define the inference contract

Purpose

The goal was to define exactly how an external application communicates with the deployed model.
The endpoint was designed for **single-image, real-time inference**.

Request Format

The client sends one JPEG or PNG image encoded as Base64:

```
{
  "image_base64": "<base64-encoded JPEG or PNG>"
}
```

Base64 allows binary image data to be transferred safely inside a JSON request.

Response Format

The endpoint returns:

```
{
  "predicted_class": "Tomato___Bacterial_spot",
  "predicted_index": 0,
  "confidence": 0.8064,
  "probabilities": {
    "...": 0.0
  }
}
```

The response contains:

- predicted class name
- predicted class index
- confidence of the prediction
- probability for every class

Important Design Decision

Class names are **not hard-coded** in the inference service.

They are reconstructed from:

```
class_to_index
```

stored inside the model checkpoint.

This prevents training and inference from accidentally using different class mappings.

What I Learned

An inference contract defines the boundary between the ML model and external applications.

The model implementation may change internally while the API contract remains stable.

Interview Explanation

I defined a simple JSON inference contract where the client sends one Base64-encoded image and receives the predicted class, confidence, class index, and probabilities for all classes.

- 7.2 Implement and test the scoring script locally

Purpose

The goal was to create reusable inference logic and the Azure ML scoring entry point.

Main Files

```
src/foliascan/inference/
├── __init__.py
└── predictor.py

infra/azure/endpoints/online/code/
└── score.py
```

Tests:

```
tests/inference/
├── test_predictor.py
└── test_score.py
```

Predictor Responsibilities

The predictor:

```
Loads best_model.pt
        ↓
Validates checkpoint contents
        ↓
Reconstructs ResNet18
        ↓
Loads weights with strict=True
        ↓
Restores class mapping
        ↓
Reuses evaluation transform
        ↓
Sets model.eval()
        ↓
Runs torch.inference_mode()
        ↓
Applies softmax
        ↓
Returns prediction response
```

Reusing Existing Training Code

Instead of duplicating training logic, inference reused:

```
create_model()
create_eval_transform()
```

This ensures the same:

- model architecture
- image resizing
- center cropping
- normalization

are used during both validation and inference.

Azure Scoring Script

Azure ML expects two important functions:

```
init()
run(raw_data)
```

init() runs when the inference container starts.

It:

- reads AZUREML_MODEL_DIR
- finds best_model.pt
- initializes the predictor once

run() executes for each inference request.

The scoring script also validates malformed JSON, missing images, invalid Base64, invalid image bytes, and checkpoint problems. The implemented score.py uses the Azure model directory and exposes these two functions.

Tests

Inference-specific tests:

```
23 passed
```

Full project tests after implementation:

```
158 passed
```

Tests covered:

- model initialization
- evaluation mode
- JPEG and PNG inference
- probability sum

- class mapping
- malformed requests
- invalid Base64
- missing model directory
- multiple checkpoints
- incompatible state dictionaries

Real Checkpoint Test

The actual GPU-trained checkpoint was also tested locally instead of relying only on mocked unit tests.

This validated the full chain:

```
Real image
→ Base64
→ score.py
→ predictor
→ real best_model.pt
→ ResNet18
→ probabilities
→ response
```

What I Learned

Unit tests validate isolated behaviour, while a real checkpoint smoke test validates integration between the saved model, preprocessing, model architecture, and inference code.

Interview Explanation

I implemented a reusable PyTorch inference module and an Azure ML scoring script with init() and run(). I reused the same model factory and evaluation transforms used during training, and tested the scoring path both with unit tests and the real trained checkpoint.

- 7.3 Define the inference environment

Purpose

The goal was to create a reproducible CPU environment containing only the dependencies required for serving predictions.

Environment

```
Name: foliascan-inference-cpu
Final version: 2
Device: CPU
Python: 3.11
```

Important packages:

```
torch==2.8.0+cpu
torchvision==0.23.0+cpu
pillow==11.3.0
pyyaml==6.0.2
azureml-inference-server-http==1.5.1
```

Why CPU?

The model required GPU acceleration for training, but a single ResNet18 image prediction is small enough to run on CPU.

Using CPU inference:

- reduces cost
- reduces deployment complexity
- avoids unnecessary GPU quota requirements

Environment Versioning

Version 1 was initially created without PyYAML.

During Docker startup, the dependency chain reached:

```
predictor.py
→ training.checkpoints
→ training.config
→ import yaml
```

and failed with:

```
ModuleNotFoundError: No module named 'yaml'
```

The missing dependency was added and a new immutable environment was registered:

`foliascan-inference-cpu:2`

instead of silently changing Version 1.

What I Learned

A production inference environment should explicitly contain **all transitive runtime dependencies**.

A dependency may not appear directly in score.py, but can still be required by imported project modules.

Environment versioning also makes deployments reproducible.

Interview Explanation

I created a lightweight CPU inference environment and versioned it in Azure ML. During local deployment testing I found a missing transitive PyYAML dependency, fixed it, and registered the corrected environment as Version 2.

- 7.4 Prepare endpoint and deployment YAML files

Purpose

The goal was to define the Azure ML online endpoint and deployment as version-controlled infrastructure.

Endpoint Definition

The endpoint configuration defines:

```
Name: foliascan-tomato-endpoint
Type: Managed online endpoint
Authentication: key
```

The endpoint represents the stable API address clients communicate with.

Deployment Definition

The deployment was named:

```
blue
```

It references:

```
Model:
foliascan-tomato-resnet18:1

Environment:
foliascan-inference-cpu:2
```

The deployment also defines:

- scoring code
- CPU instance type
- instance count
- environment variables
- model
- environment

Endpoint vs Deployment

A useful distinction:

```
Endpoint
    ↓
Stable API address
    ↓
Traffic routing
```

```
    ↓
Deployment
    ↓
Model + environment + code + compute
```

An endpoint can theoretically route traffic among multiple deployments:

```
blue
green
canary
```

PYTHONPATH Issue

During the first Docker deployment, Azure mounted the repository under:

```
/var/azureml-app/foliascan-azure
```

but the initial environment variable was:

```
/var/azureml-app/src
```

This caused:

```
ModuleNotFoundError: No module named 'foliascan'
```

The correct value became:

```
/var/azureml-app/foliascan-azure/src
```

What I Learned

Container paths can differ from local filesystem paths.

A project using a src/ layout must ensure that the package directory is importable inside the deployed container.

Interview Explanation
I defined the managed endpoint and deployment using YAML. The deployment explicitly referenced the registered model and inference environment, and I configured the container Python path so the project package could be imported correctly.

- 7.5 Test the deployment locally with Docker

Purpose

The goal was to identify deployment problems **before creating paid Azure compute**.

Azure ML's local deployment mode created a Docker container resembling the cloud inference environment.

Problems Found

Problem 1 — Package Import

Error:

```
ModuleNotFoundError: No module named 'foliascan'
```

Cause:

```
Incorrect PYTHONPATH
```

Fix:

```
/var/azureml-app/foliascan-azure/src
```

Problem 2 — Missing Dependency

After fixing the import path:

```
ModuleNotFoundError: No module named 'yaml'
```

Cause:

```
PyYAML missing from inference environment
```

Fix:

```
pyyaml==6.0.2
```

and environment Version 2.

Successful Local Deployment

After the fixes:

```
Provisioning state: Succeeded
```

The endpoint successfully returned a real prediction.

The local result was approximately:

```
Predicted class: Tomato___Bacterial_spot
Predicted index: 0
Confidence: ~0.8064
```

Cleanup

The local deployment/container and Docker image were deleted after validation.

What I Learned

Local container testing is extremely useful because it exposes issues that normal Python tests cannot find, such as:

- container filesystem layout
- missing runtime packages
- startup behaviour
- inference-server configuration
- environment variables

Interview Explanation

Before deploying to Azure, I tested the complete deployment locally with Docker. This exposed an incorrect container Python path and a missing runtime dependency, which I fixed before creating paid cloud compute.

- 7.6 Create and invoke the Azure endpoint

Purpose

The goal was to deploy the registered model to a real Azure ML Managed Online Endpoint and verify remote inference.

This step produced several real Azure infrastructure issues.

Resource Provider Issue

Initial endpoint creation failed with:

```
SubscriptionNotRegistered
```

Required Azure resource providers were inspected and registered.

Examples included:

```
Microsoft.MachineLearningServices
Microsoft.PolicyInsights
Microsoft.Cdn
Microsoft.Compute
Microsoft.Network
Microsoft.Storage
Microsoft.KeyVault
Microsoft.ManagedIdentity
Microsoft.ContainerRegistry
Microsoft.Insights
Microsoft.OperationalInsights
Microsoft.Quota
```

After registration, endpoint creation succeeded.

Endpoint Result

```
foliascan-tomato-endpoint
Provisioning state: Succeeded
Authentication: key
Region: Norway East
```

Compute Quota Problem

The initial deployment used:

```
Standard_DS3_v2
```

which has 4 vCPUs.

Managed Online Endpoints require additional quota for rolling upgrades/recovery.

For one DS3_v2 instance:

```
ceil(1.2 × 1) × 4 = 8 vCPUs
```

But Azure ML showed:

```
Standard DSv2 Family Cluster Dedicated vCPUs

Current: 0
Limit: 6
```

Therefore the deployment failed with:

```
OutOfQuota
```

Important Discovery

There were two different quota concepts:

```
General Azure regional quota
```

and:

```
Azure ML VM-family dedicated-cluster quota
```

The general regional limit was sufficient:

```
Total Regional vCPUs: 10
```

but the Azure ML DSv2 family quota was:

```
6
```

Solution

Instead of requesting more quota, the deployment was changed to:

```
Standard_DS2_v2
```

with 2 vCPUs.

Required quota:

```
ceil(1.2 × 1) × 2 = 4 vCPUs
```

which fit within the existing limit of 6.

Final cloud deployment:

```
Deployment: blue
State: Succeeded
Instance type: Standard_DS2_v2
Instance count: 1
```

Real Cloud Inference

The previously generated JSON request was sent to the actual Azure endpoint.

The container logs confirmed:

```
POST /score 200
```

meaning:

```
request reached endpoint
→ scoring code executed
→ model inference succeeded
→ HTTP 200 response returned
```

The Azure logs also confirmed successful system setup, model download, container startup, and readiness.

What I Learned

Cloud deployments can fail even when:

- the code is correct
- Docker tests pass
- the environment works

because deployment also depends on:

- subscription configuration
- resource providers
- region
- compute quotas
- Azure ML-specific quotas
- VM SKU selection

Interview Explanation

I deployed the model to an Azure ML Managed Online Endpoint. During deployment I diagnosed both subscription provider registration and Azure ML VM-family quota problems. I changed the endpoint compute from DS3_v2 to DS2_v2 to fit the available quota and successfully completed a real cloud inference request.

- 7.7 Inspect logs, document the endpoint, and delete it

Purpose

The goal was to verify the final cloud deployment, preserve evidence of the successful deployment, and remove paid resources.

Verification

The deployment reached:

```
State: Succeeded
```

Cloud logs showed:

```
SystemSetup: Succeeded
UserContainerImagePull: Succeeded
ModelDownload: Succeeded
UserContainerStart: Succeeded
ContainerReady
POST /score 200
```

These confirm that:

- environment image was available
- registered model was downloaded
- scoring container started
- health checks passed
- inference request succeeded

Metadata Saved

Before deletion, metadata was stored under:

```
artifacts/azure/endpoints/
└── foliascan-tomato-endpoint/
    ├── endpoint-metadata.json
    ├── deployment-metadata.json
    └── inference-response.json
```

These files are not intended for Git because they may contain internal Azure resource information.

Cost Cleanup

After successful inference, the Managed Online Endpoint was deleted:

```
foliascan-tomato-endpoint
```

A final endpoint listing returned no rows.

This confirms that the continuously running deployment compute was removed.

Git

The final cloud-compatible compute configuration was:

```
Standard_DS2_v2
```

and the implementation was merged through:

```
PR #16 — Feat/online inference
```

What I Learned

Deployment is not complete when the API merely returns a prediction.

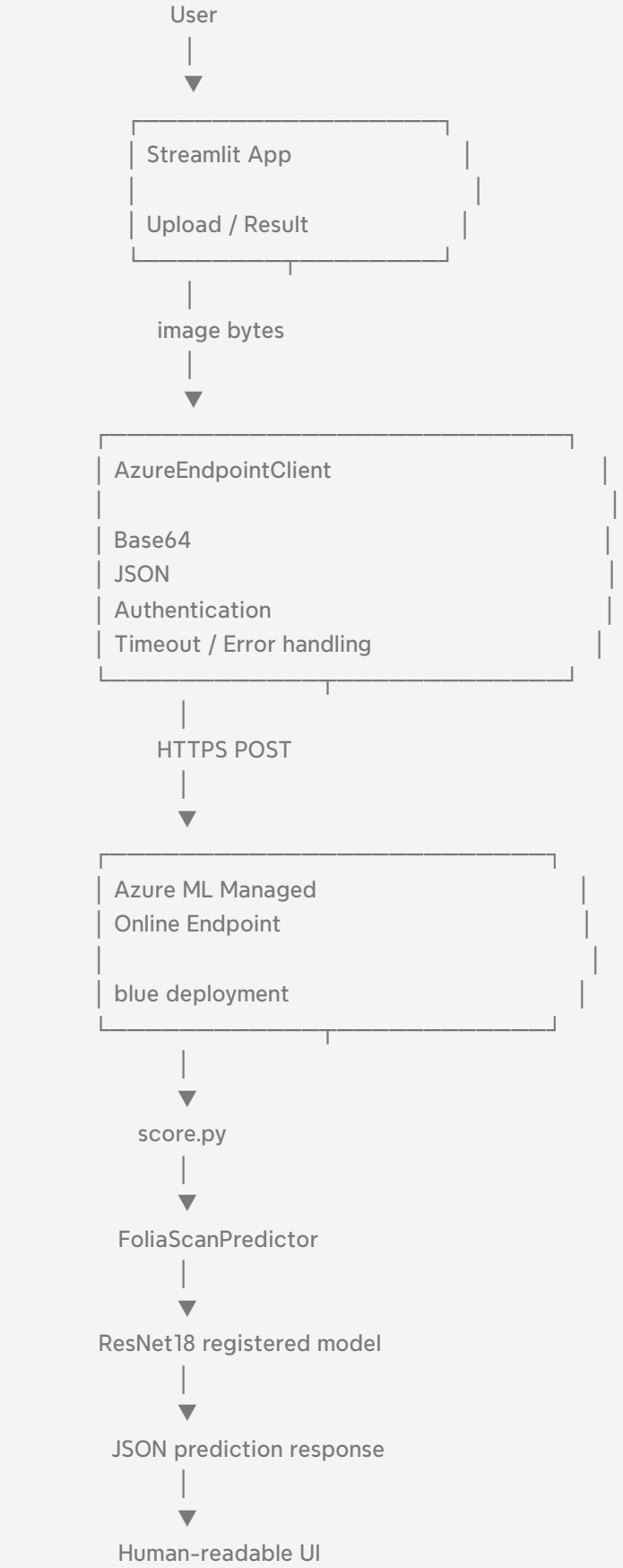
A responsible cloud workflow also includes:

```
Verify
→ collect evidence
→ inspect logs
→ control costs
→ clean up resources
```

Interview Explanation

After validating the real cloud prediction, I inspected the deployment logs, saved endpoint and deployment metadata for traceability, and deleted the managed endpoint immediately to avoid unnecessary ongoing compute cost.

Phase 8 — Add a simple application



Concept | What I learned

Client application | Provides a usable interface around an ML service
Separation of concerns | UI and endpoint communication should be separate
HTTPS inference | Azure ML endpoints can be consumed like normal APIs
Environment variables | Keep endpoint configuration and secrets outside code
Base64 | Allows image bytes to be transported inside JSON
Timeout | Prevents requests from waiting indefinitely
Response validation | Never assume an external API returned valid data
Mocking | Allows cloud integrations to be tested without cloud resources
Presentation layer | Converts machine-friendly outputs into user-friendly results
Streamlit | Fast way to build an ML demo/application
Integration test | Validates communication between multiple components
End-to-end test | Validates the entire real user-to-model workflow
Branding | Makes a portfolio application more coherent and professional
Documentation | Makes architecture and decisions understandable to others
Secret management | Credentials must never be committed or displayed
Cost cleanup | Temporary cloud resources should be removed after validation

I built a Streamlit client application around my Azure ML deployment, keeping the UI separate from a reusable HTTPS endpoint client. The client securely reads endpoint configuration from environment variables, Base64-encodes uploaded images, handles authentication, timeouts, network failures, and response validation. I tested the client extensively with mocked HTTP requests and then performed a real end-to-end test from Streamlit to the Azure ML Managed Online Endpoint. Finally, I improved the UI with human-readable disease labels and FoliaScan branding, documented the workflow and architecture, and deleted the temporary cloud endpoint after validation to control cost.

- 8.1 Define the client application architecture

Purpose

The goal was to build a lightweight application that allows a user to interact with the deployed FoliaScan model without needing Azure CLI or Python scripts.

The application needed to:

- accept one tomato-leaf image
- send it to the Azure ML endpoint
- receive the model prediction
- present the result in a readable form

Architecture

The application was intentionally separated into two layers:

```
Streamlit UI
     ↓
Reusable endpoint client
     ↓
Azure ML Managed Online Endpoint
     ↓
ResNet18 model
```

The UI does not contain Azure HTTP logic directly.

Instead:

```
app/streamlit_app.py
        ↓
src/foliascan/client/azure_endpoint.py
```

Why This Separation Matters

Keeping the endpoint client separate from Streamlit makes the application:

- easier to test
- easier to maintain
- reusable from another application later
- independent from a specific UI framework

For example, the same client could later be reused from:

```
FastAPI
CLI
another Python application
different frontend
```

without rewriting the endpoint communication logic.

Configuration

The application reads Azure endpoint configuration from environment variables:

```
FOLIASCAN_ENDPOINT_URL
FOLIASCAN_ENDPOINT_KEY
```

Credentials are not hard-coded in source code.

What I Learned

A client application should separate:

```
presentation logic
from
service/integration logic
```

This improves maintainability, testability, and security.

Interview Explanation

I separated the Streamlit presentation layer from a reusable Azure endpoint client. This allowed me to test the HTTP integration independently and kept credentials and Azure communication logic out of the UI code.

- 8.2 Implement reusable Azure endpoint client

Purpose

The goal was to implement a small Python client capable of calling the Azure ML Managed Online Endpoint.

The client uses ordinary HTTPS rather than the Azure SDK.

Request Flow

The client receives raw image bytes:

```
JPEG / PNG bytes
      ↓
Base64 encoding
      ↓
JSON request
      ↓
HTTPS POST
      ↓
Azure ML endpoint
```

Request contract:

```
{
  "image_base64": "<base64 image>"
}
```

This is the same inference contract established in Phase 7.

Authentication

The request includes:

```
Authorization: Bearer <endpoint-key>
Content-Type: application/json
```

The endpoint key is loaded only from the environment.

It is not:

- printed
- logged
- stored in the repository
- exposed to Streamlit

HTTP Timeout

An explicit timeout was used:

```
30 seconds
```

This prevents the client from waiting indefinitely if the endpoint becomes unavailable.

Response Validation

The response must contain:

```
predicted_class
predicted_index
confidence
probabilities
```

The client validates the returned JSON before passing it to the UI.

Error Handling

Application-level errors were implemented for:

```
Missing endpoint URL
Missing endpoint key
Empty image
Network failure
Request timeout
HTTP 4xx / 5xx
Invalid JSON response
Missing prediction fields
Invalid prediction fields
```

This means infrastructure/network errors do not appear as unexplained Python exceptions to the user.

Important Design Decision

The endpoint client does **not require the Azure SDK** to perform inference.

The Azure ML endpoint is simply consumed as an HTTPS API.

This keeps the client lightweight and reduces coupling to Azure-specific libraries.

Tests

HTTP calls were mocked, so client tests did not require a live Azure endpoint.

Tests covered:

```
Successful request
Base64 payload
Authorization header
HTTP timeout
Missing configuration
Empty images
Network errors
Timeout errors
HTTP errors
Invalid JSON
Invalid/missing response fields
```

What I Learned

A production-facing ML application needs more than just model inference.

The API client must also handle:

```
authentication
serialization
timeouts
network failures
response validation
error translation
```

Interview Explanation

I implemented a reusable HTTPS client for the Azure ML endpoint. It Base64-encodes image data, authenticates using an endpoint key loaded from environment variables, validates prediction responses, uses explicit timeouts, and provides clear application-level errors.

- 8.3 Build the Streamlit user interface

Purpose

The goal was to provide a simple visual interface for interacting with the model.

Streamlit was selected because the project focuses on:

```
AI engineering
Azure ML
deployment
MLOps
```

rather than frontend engineering.

It allowed us to build a useful interface without introducing unnecessary React/backend complexity.

Main User Flow

The application follows one simple workflow:

```
Upload tomato leaf
       ↓
Preview image
       ↓
Analyze
       ↓
Wait for inference
       ↓
Display prediction
```

UI Components

The application includes:

- FoliaScan branding/logo
- JPEG/PNG uploader
- image preview
- Analyze button
- loading spinner
- predicted disease/class
- confidence percentage
- class probabilities
- educational disclaimer

Logo and Branding

Project assets were organized under:

```
assets/
├── branding/
├── roadmap/
└── screenshots/
```

The Streamlit application uses:

```
assets/branding/foliascan-logo.png
assets/branding/foliascan-icon.png
```

The logo provides the main brand identity, avoiding unnecessary duplication of the FoliaScan name in the page header.

Human-Readable Class Names

The model internally returns labels such as:

```
Tomato___Late_blight
Tomato___Bacterial_spot
```

These are useful machine-readable class identifiers but are not ideal for users.

The UI therefore converts them to:

```
Late Blight
Bacterial Spot
```

This transformation happens only in the **presentation layer**.

The backend inference contract remains unchanged.

Prediction Presentation

Instead of showing only the raw prediction, the polished interface displays:

```
Predicted class
Confidence
Top 3 probabilities
All probabilities
```

The full probability table remains available for more detailed inspection.

Important Design Principle

Backend representation:

```
Tomato___Late_blight
```

User representation:

```
Late Blight
```

The UI improves readability without altering model behaviour.

What I Learned

The output of an ML model should not automatically become the final user interface.

There is usually a presentation layer between:

```
machine representation
and
human-readable result
```

Interview Explanation

I built a lightweight Streamlit interface around the model endpoint. I kept the workflow focused on image upload and prediction, added FoliaScan branding, converted internal class identifiers into human-readable labels, and presented confidence and probability information clearly.

- 8.4 Add tests and local validation

Purpose

The goal was to validate the application without creating Azure resources or paying for endpoint compute.

Mocked Client Testing

HTTP interactions were mocked.

This made the tests:

- fast
- deterministic
- free
- independent of Azure availability

Presentation Tests

Additional lightweight tests verified presentation logic such as:

```
Tomato___Late_blight
→
Late Blight
```

and correct sorting/limiting of probability results.

Validation Results

After the application polish:

```
Client tests: 20 passed
Full project tests: 178 passed

Ruff: passed
mypy: passed
git diff --check: passed
```

The Streamlit module could also be imported without triggering an Azure request.

This was important because:

```
import application
```

should not automatically:

```
connect to Azure
make predictions
consume resources
```

Local UI Validation

The application was started locally using:

```
poetry run streamlit run app/streamlit_app.py
```

The following were tested before creating a cloud endpoint:

```
logo
uploader
image preview
button
layout
disclaimer
configuration errors
```

Configuration Error Test

Because the Azure endpoint had already been deleted after Phase 7, pressing Analyze initially produced:

```
FOLIASCAN_ENDPOINT_URL is not configured.
```

This was expected and demonstrated that configuration error handling worked correctly.

What I Learned

Not every application test should require cloud infrastructure.

A good testing strategy separates:

```
unit tests
integration tests
local UI tests
real cloud end-to-end tests
```

Interview Explanation

I tested the endpoint client with mocked HTTP responses so the normal test suite did not depend on Azure. I also validated presentation helpers and confirmed the Streamlit application could start and import without making a cloud request.

- 8.5 Test the app against the real Azure endpoint

Purpose

The goal was to verify the complete application path using a real Azure ML deployment.

This was the final integration test:

```
User
 ↓
Streamlit
 ↓
Endpoint client
 ↓
HTTPS
 ↓
Azure ML Managed Online Endpoint
 ↓
ResNet18 model
 ↓
Prediction
 ↓
Streamlit
```

Temporary Endpoint Recreation

Because the endpoint from Phase 7 had been deleted to avoid ongoing charges, it was temporarily recreated.

The existing version-controlled configuration was reused:

```
Endpoint:
foliascan-tomato-endpoint

Deployment:
blue

Compute:
Standard_DS2_v2
```

The deployment successfully reached:

```
Succeeded
```

with:

```
blue: 100% traffic
```

Runtime Configuration

The endpoint URL and key were retrieved from Azure and assigned only to the local PowerShell environment:

```
FOLIASCAN_ENDPOINT_URL
FOLIASCAN_ENDPOINT_KEY
```

The values were not written to the repository.

Real End-to-End Prediction

A real tomato-leaf image was uploaded through the Streamlit UI.

The application successfully returned:

```
Prediction:
Tomato___Late_blight

Displayed as:
Late Blight

Confidence:
85.3%
```

Other high-ranking predictions included:

```
Target Spot: 13.4%
Early Blight: 0.7%
```

This demonstrated that all layers worked together:

```
image upload
Base64 conversion
authentication
HTTP request
Azure routing
container scoring
PyTorch inference
JSON response
UI rendering
```

Important Distinction

The result represents the model's prediction.

It does **not** prove that the plant actually has Late Blight.

This is why the application contains an educational disclaimer.

What I Learned

A successful unit test is different from a successful real end-to-end system test.

The real test validated interfaces between:

```
browser
Streamlit
Python client
Azure authentication
Azure endpoint
container
model
response
```

Interview Explanation

I performed a real end-to-end test by temporarily recreating the Azure ML endpoint and running inference through the Streamlit application. The complete path from uploaded image to cloud model prediction worked successfully.

- 8.6 Document the application and final workflow

Purpose

The final subphase focused on turning the functional prototype into a presentable portfolio project.

The objective was not to add more features, but to improve:

```
presentation
documentation
branding
usability
traceability
```

UI Polish

The final application includes:

- FoliaScan logo
- FoliaScan page icon
- concise introductory copy
- restrained green visual styling
- controlled image preview size
- prominent prediction section
- human-readable disease names
- confidence percentage
- Top 3 predictions
- expandable full probability table
- subtle educational disclaimer

The design intentionally remained simple rather than becoming a complex dashboard.

Project Assets

Static assets are organized as:

```
assets/
├── branding/
│   ├── foliascan-logo.png
│   └── foliascan-icon.png
│
├── roadmap/
│   ├── foliascan-learning-roadmap.png
│   ├── foliascan-learning-roadmap.pdf
│   └── foliascan-learning-roadmap.xmind
│
└── screenshots/
    └── streamlit-cloud-inference.png
```

README Improvements

The README was updated to represent the actual current project rather than the earlier Phase 4 state.

It now covers:

```
Project overview
Key features
Architecture
Model results
Azure ML workflow
Streamlit application
Learning roadmap
Learning materials
Project phases
Repository structure
Local setup
Azure configuration
Security
Cost considerations
Disclaimer
```

Application Screenshot

The successful real inference screenshot is included in the README as evidence of the completed end-to-end system.

Learning Roadmap

The phase-by-phase roadmap is displayed directly in the README:

```
foliascan-learning-roadmap.png
```

The user can also access:

```
PDF learning notes
editable XMind roadmap
```

This documents not only the final project but also the learning process used to build it.

Client Documentation

A dedicated document was added:

```
docs/client-application.md
```

It explains:

- client architecture
- environment variables
- inference flow
- error handling
- local execution
- real Azure validation
- security considerations
- endpoint cleanup

Security

The application and documentation avoid exposing:

```
Azure endpoint keys
subscription IDs
endpoint URLs
Base64 image payloads
sensitive Azure resource metadata
```

Cost Control

After successful end-to-end validation, the Managed Online Endpoint was deleted.

The endpoint list returned no matching resources.

Temporary environment variables were also removed:

```
FOLIASCAN_ENDPOINT_URL
FOLIASCAN_ENDPOINT_KEY
```

This is important because managed endpoint compute continues to incur cost while deployed.

What I Learned

Completing an ML engineering project includes more than creating a working model.

A portfolio-quality system also needs:

```
clear architecture
usable interface
documentation
security
cost awareness
```

```
reproducibility
evidence of validation
```

Interview Explanation

I finished the application by improving the Streamlit presentation, adding project branding, documenting the architecture and inference workflow, and updating the README with the real application screenshot and learning roadmap. After end-to-end validation, I deleted the Azure endpoint to prevent ongoing compute costs.

Phase 9 — Add GitHub Actions

Concept	What I learned
CI	Automatically validates changes continuously
GitHub Actions	GitHub's workflow automation platform
Workflow	YAML definition of automated jobs
Trigger	Event that starts a workflow
Runner	Temporary machine where CI commands execute

Job | Group of workflow steps executed on a runner
Step | Individual action or shell command
Quality gate | Check that must pass for CI to succeed
Dependency caching | Reuses downloaded dependencies across runs
poetry.lock | Provides reproducible dependency versions
Least privilege | Give CI only the permissions it needs
Smoke test | Lightweight check that a component can start/import
PR validation | Tests changes before they enter main
Push validation | Checks the final state after merge
CI vs CD | CI validates code; CD automates release/deployment
Cost boundary | Keep expensive cloud work out of routine CI
Secrets management | Do not expose credentials when they are unnecessary

I added a GitHub Actions CI pipeline to automatically validate FoliaScan on pull requests and pushes to main. The workflow runs on a GitHub-hosted Ubuntu runner with Python 3.11 and Poetry 2.4.1, uses dependency caching based on the lock file, and executes the full pytest suite, Ruff linting, mypy type checking, and a Streamlit import smoke test. I validated the workflow through a real pull request and deliberately excluded Azure authentication, training, and endpoint deployment from CI to keep automated checks secure, deterministic, and free from accidental cloud compute costs.

- 9.1 Define the CI strategy

Purpose

The goal was to automate the quality checks that had previously been executed manually before every commit or pull request.

Before CI, the development workflow included commands such as:

```
pytest
ruff
mypy
Streamlit import check
```

These checks depended on the developer remembering to run them.

With Continuous Integration:

```
Code change
    ↓
Pull Request
    ↓
GitHub Actions
    ↓
Automated validation
    ↓
Pass / Fail
```

CI Scope

The workflow was designed for **code quality validation**, not cloud deployment.

It runs on:

```
Pull request → main
Push → main
```

This provides two levels of verification:

```
Feature branch
      ↓
Pull Request
      ↓
CI validates proposed change

Merge
      ↓
main
      ↓
CI validates final repository state
```

Important Design Decision

The CI workflow intentionally does not:

```
Authenticate to Azure
Create Azure resources
Start Azure compute
Train models
Deploy managed endpoints
Invoke cloud inference
Use Azure secrets
```

This keeps CI:

- inexpensive
- deterministic
- safe
- fast enough for normal development
- independent of Azure service availability

What I Learned

Continuous Integration does not necessarily mean deploying software.

CI primarily answers:

"Does this code still meet the project's quality requirements?"

Deployment automation would belong to **Continuous Delivery / Continuous Deployment (CD)**.

Interview Explanation

I designed the FoliaScan CI pipeline to validate code quality on every pull request and push to main. I deliberately kept Azure training and deployment outside CI to avoid accidental cloud resource creation, cost, and unnecessary dependencies on external infrastructure.

- 9.2 Create the GitHub Actions workflow

Purpose

The goal was to define a reproducible CI pipeline using GitHub Actions.

GitHub Actions workflows are stored under:

```
.github/workflows/
```

FoliaScan uses:

```
.github/workflows/ci.yml
```

Workflow Structure

The workflow contains:

```
Workflow
  ↓
Job
  ↓
Steps
```

For FoliaScan:

```
CI
 └── Quality checks
      ├── Checkout repository
      ├── Install Poetry
      ├── Set up Python
      ├── Install dependencies
      ├── Run pytest
      ├── Run Ruff
      ├── Run mypy
      └── Verify Streamlit import
```

Trigger Configuration

The workflow runs for:

```
push:
  branches:
    - main

pull_request:
  branches:
    - main
```

This means feature-branch pushes alone do not trigger the workflow unless a pull request

targeting main exists.

Runner

The workflow uses:

```
ubuntu-latest
```

GitHub creates a temporary Linux runner for each job.

Conceptually:

```
GitHub-hosted Ubuntu VM
        ↓
checkout repository
        ↓
install runtime/tools
        ↓
run validation
        ↓
destroy runner
```

This runner is separate from:

- the local Windows machine
- Azure ML compute
- Azure endpoint VMs

Timeout

The job uses:

```
timeout-minutes: 15
```

This prevents a broken or hanging command from consuming runner time indefinitely.

Permissions

The workflow uses:

```
contents: read
```

It only needs permission to read the repository.

This follows the principle of **least privilege**.

What I Learned

A GitHub Actions YAML file defines:

```
when automation runs
where it runs
what permissions it has
what commands it executes
```

Interview Explanation

I created a GitHub Actions workflow under .github/workflows/ci.yml using a GitHub-hosted Ubuntu runner. It runs automatically for pull requests and pushes to main, uses read-only repository permissions, and has a timeout to prevent stuck jobs.

- 9.3 Add dependency caching and reproducibility

Purpose

Each GitHub Actions run starts from a mostly clean environment.

Without caching:

```
CI run
  ↓
download dependencies
  ↓
install dependencies

next CI run
  ↓
download them again
```

Dependency caching reduces repeated downloads.

Python Version

The project uses:

```
Python 3.11
```

Locally, the Poetry environment was verified as:

```
Python 3.11.3
```

The GitHub workflow explicitly configures:

```
Python 3.11
```

This is important because the machine's system Python version may be different.

For example, the local Windows system had:

```
Python 3.14.5
```

but the project itself remained on Python 3.11.

Poetry Version

Poetry was also pinned:

```
Poetry 2.4.1
```

So the project uses the same Poetry version locally and in CI.

Dependency Locking

Dependencies are installed from:

```
pyproject.toml
+
poetry.lock
```

The lock file makes dependency resolution much more reproducible.

Poetry Cache

Caching was enabled with:

```
cache: "poetry"
cache-dependency-path: poetry.lock
```

Conceptually:

```
poetry.lock unchanged
       ↓
reuse dependency cache

poetry.lock changed
       ↓
new cache state
```

Important Point

Caching does **not replace**:

```
poetry install
```

The install step still runs.

The cache simply reduces repeated network downloads and setup work.

What I Learned

Reproducibility depends on more than source code.

A stable CI environment should also control:

```
Python version
Package manager version
Dependency lock file
Dependency installation
```

Interview Explanation

I pinned Python 3.11 and Poetry 2.4.1 in CI and enabled Poetry dependency caching using poetry.lock. This makes the workflow more reproducible while reducing repeated dependency downloads between runs.

- 9.4 Run automated quality checks

Purpose

The CI pipeline should reproduce the important local quality checks automatically.

Full Test Suite

The workflow runs:

```
poetry run pytest
```

At the end of development, the FoliaScan test suite contained:

```
178 tests
```

covering areas such as:

```
client
cloud integration helpers
dataset processing
evaluation
inference
training
MLflow
Streamlit presentation
```

Ruff

The workflow runs:

```
poetry run ruff check .
```

Ruff checks code-quality issues such as:

- unused imports
- import ordering
- style problems
- selected Python lint rules

mypy

The workflow runs:

```
poetry run mypy
```

mypy performs static type checking.

This can detect problems before runtime, such as incompatible types or missing type information in important paths.

Streamlit Import Smoke Test

The workflow also verifies:

```
import app.streamlit_app
```

The important requirement is:

```
Importing the UI must not automatically call Azure.
```

This ensures that simply loading the application module does not:

- invoke an endpoint
- require credentials
- create cloud usage

Quality Gates

The CI effectively establishes:

```
Tests pass
AND
Ruff passes
AND
mypy passes
AND
Streamlit imports
        ↓
CI success
```

If any command returns a non-zero exit code:

```
CI fails
```

What I Learned

CI quality gates turn development conventions into automated rules.

Instead of saying:

"Developers should remember to run the tests."

the repository enforces:

"The pull request visibly fails if the tests do not pass."

Interview Explanation

The CI workflow runs the full pytest suite, Ruff linting, mypy type checking, and a Streamlit import smoke test. Any failure causes the GitHub Actions job to fail, so code quality is checked automatically before changes are merged.

- 9.5 Validate the workflow on a real Pull Request

Purpose

A workflow file that looks correct locally is not enough.

The goal was to validate it using an actual GitHub pull request and a real GitHub-hosted runner.

Real PR Validation

The CI feature was submitted through a pull request.

GitHub automatically detected the workflow and started:

```
CI / Quality checks
```

The result was:

```
All checks have passed ✅
```

The first real workflow run took approximately:

```
3 minutes
```

Why This Test Matters

Before this test, we knew:

```
YAML parses locally
commands work locally
```

After the real PR test, we additionally knew:

```
GitHub recognizes workflow
trigger works
Ubuntu runner works
Python setup works
Poetry setup works
dependencies install on a clean machine
tests work on Linux
Ruff works
mypy works
Streamlit import works
```

This is a much stronger validation.

Push-to-Main Validation

After merging the PR, another run was triggered because the workflow also listens to:

```
push → main
```

This validated the final merged state.

Subsequent Pull Requests

After CI was introduced, later changes such as branding updates also automatically triggered the workflow.

This demonstrates that CI became part of the normal repository workflow rather than being a one-time test.

What I Learned

Passing tests on a developer machine does not guarantee that the project works in a clean environment.

A clean CI runner detects hidden dependencies such as:

```
local files
machine-specific configuration
missing dependencies
OS assumptions
uncommitted resources
```

Interview Explanation

I validated the pipeline using a real pull request. GitHub created a clean Ubuntu runner, installed the project, and successfully completed all quality checks. After the workflow was merged, it automatically validated subsequent pull requests and pushes to main as well.

- 9.6 Add CI status/documentation and finalize the project

Purpose

The final step was to make the CI design understandable to other developers and recruiters viewing the repository.

README CI Badge

A GitHub Actions status badge was added near the top of the README.

Conceptually:

```
CI | passing
```

The badge gives an immediate indication that automated validation exists and the latest workflow is healthy.

Continuous Integration Section

The README documents:

```
workflow triggers
Python version
Poetry version
pytest
Ruff
mypy
Streamlit smoke test
dependency caching
Azure exclusions
```

Dedicated CI Documentation

A dedicated file was added:

```
docs/ci.md
```

It explains:

- CI purpose
- workflow triggers
- GitHub-hosted runner
- dependency installation
- dependency caching
- automated quality gates

- repository permissions
- security boundaries
- cost considerations
- real pull-request validation

Security Documentation

The documentation explicitly states that CI does not use:

```
Azure credentials
Azure subscription identifiers
Azure endpoint keys
cloud deployment permissions
```

This is important because CI systems often have access to sensitive secrets.

In FoliaScan, they are unnecessary.

Cost Awareness

Azure operations remain outside CI.

This prevents ordinary code changes from accidentally triggering:

```
GPU training
managed endpoint deployment
paid Azure compute
```

Project Status

Phase 9 represents the final planned development phase.

The completed project now covers:

```
local ML development
        ↓
dataset engineering
        ↓
Azure infrastructure
        ↓
Azure training
        ↓
MLflow tracking
        ↓
model registration
        ↓
online inference
        ↓
Streamlit application
```

```
        ↓
GitHub Actions CI
```

What I Learned

Documentation is part of engineering.

A CI pipeline should make clear:

```
what is automated
what is not automated
why those boundaries exist
how failures should be interpreted
```

Interview Explanation

I finalized the project by documenting the GitHub Actions pipeline in both the README and dedicated CI documentation. I also added a CI status badge and clearly documented the security and cost boundary that keeps Azure resource creation outside automated CI.